Technische
Hochschule
Rosenheim

Faculty of computer science

Course of study Master of computer science

Wi-Fi for time critical applications: Practical measurements and evaluation of influencing variables on real time capability

Master Thesis

by

Jakob Hasche

Date of submission: tt.mm.jjjj
First examiner: Prof. Dr. Wolfgang Mühlbauer
Second examiner: Prof. Dr. Florian Künzner

Eigenständigkeitserklärung / Declaration of Originality

Hiermit bestätige ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken (dazu zählen auch Internetquellen) entnommen sind, wurden unter Angabe der Quelle kenntlich gemacht.

I declare that I have authored this thesis independently, that I have not used other than the declared sources / resources, and that I have explicitly marked all material which has been quoted either literally or by content from the used sources.

Rosenheim, the September 1, 2025

Jakob Hasche

Abstract

text

Schlagworte:

List of abbreviations

TCP Transmission Control Protocol

UDP User Datagram Protocol

rt-data real-time data

non-rt-data non-real-time data

rt-system real-time system

rt real-time

SCADA Supervisory Control and Data Acquisition

OS Operating System

RTOS Real Time Operating System

AP Access Point

BPF Berkley Package Filter

eBPF extended Berkley Package Filter

RTT Round-Trip-Time

BTF BPF Type Format

CO-RE Compile Once - Run Everywhere

JIT Just-In-Time

CPU Central Processing Unit

kprobe Kernel Probe

uprobe Userspace Probe

kretprobe Kernel Return Probe

uretprobe Userspace Return Probe

TDP Thermal Design Power

USDT User-level statically defined tracing

WCET Worst-case execution time

DFSM Deterministic Finit State Machine

TCP Transmission Control Protocol

NTP Network Time Protocol

PTP Precision Time Protocol

Qdisc Queueing Discipline

DMA Direct Memory Access

NIC Network Interface Card

OSI-model Open Systems Interconnection model

ABI Application Binary Interface

LAN Local Area Network

PID	Process Id
QoS	Quality of Service
WPA	Wi-Fi Protected Access
RegEx	Regular Expression
HE	High Efficiency
VHT	Very High Throughput
HT	High Throughput

Contents

List of Figures

List of Tables

Listings

1 Introduction

1.1 Problem description

1.2 Objective

1.3 Motivation

1.4 Layout of the work

2 Basics

2.1 Used Hardware

2.2 Yocto Linux

2.3 Linux Software Stack

2.4 Wi-Fi standards

2.4.1 Quality of Service

2.4.2 Scheduling

2.4.3 IEEE 802.11n

2.4.4 IEEE 802.11ac

2.4.5 IEEE 802.11ax

2.4.6 IEEE 802.11be

2.5 Medium Access Mechanisms

2.6 Disruptive factors in Wi-Fis

2.7 Real Time

2.7.1 Real Time in general

2.7.2 Real Time Kernel

2.7.3 Real Time Capability of Wi-Fi

3 Related Work

3.1 RT-Wi-Fi

3.2 Quality of Service in Wi-Fi

3.3 Disruptive factors in Wi-Fi's

3.4 Categorization of this work

4 Test Environment

In this chapter, the test environment is examined in more detail. In the following, the terms test environment, test system and Operating System (OS) are used. The test environment comprises both the test system and the OS. The test system refers exclusively to the applications that are part of the test environment and run on top of the OS. Accordingly, the test system does not include the underlying hardware or the OS itself.

This chapter is divided into the three parts requirements, test system and the OS. A detailed analysis of the requirements is provided at the outset, as they serve as the basis for both the development of the OS and the test concept. Thus, the requirements constitute the foundation for the overall design of the test environment. Afterwards the OS, as it forms the foundation on which the test system will later be implemented, will be described. Finally, the test system itself will be discussed in more detail.

4.1 Requirements

In this section, the requirements of the test environment are analyzed in greater detail. The discussion begins with a description of the system architecture, followed by an outline of general real-time (rt) requirements. These are then used to define the specific requirements applicable to the test system.

The test environment is illustrated in Figure 4.1. It consists of two Raspberry Pi 5 devices, whose exact composition is described in detail in Chapter 2.1. One Raspberry Pi acts as the server, while the other functions as the client. Both devices run the same OS.

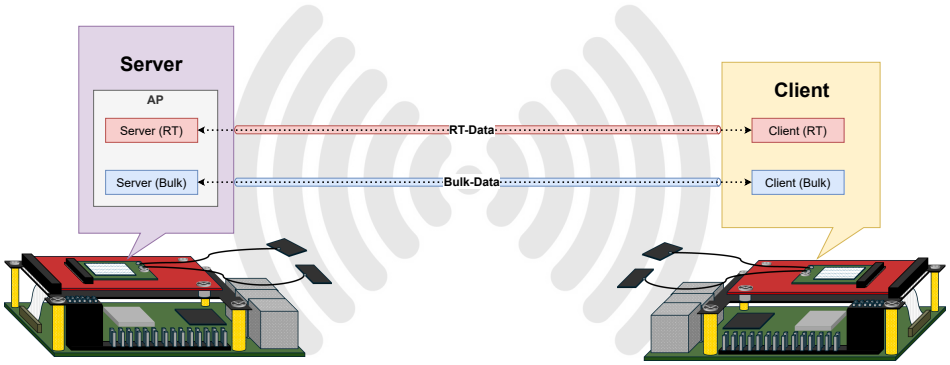

Figure 4.1 Outline of the overall test environment

There are two communication links between the server and the client. One link which transfers real-time data (rt-data) and another one for transferring non-real-time data (non-rt-data). These two links are illustrated in Figure 4.1 in blue and red. These connections are handled by separate processes but operate within the same network. The server provides an access point to which the client connects, thereby establishing a peer-to-peer connection.

In summary, the goal of the test environment is to transmit both rt-data and non-rt-data data over the same wireless network interface. While the bulk data throughput should be maximized, the primary focus is to ensure that no rt violations occur on the rt connection.

Since the test system is intended to transmit rt-data among normal data, it must fulfill the requirements of a real-time system (rt-system). Therefore, the following section first outlines the general requirements of rt-systems. Subsequently, additional requirements that go beyond standard rt criteria and are specifically relevant for this system within the given test environment are discussed in more detail.

4.1.1 Requirements Real Time

To enable a system to achieve rt capability, numerous factors must be considered. Since this work aims to evaluate the rt capabilities of Wi-Fi, an interface that is not inherently rt capable—and given that hardware-based requirements would have little impact in this context while significantly broadening the scope of this thesis, only software-related requirements will be addressed.

As previously mentioned in Chapter 2.7, the primary objective of a rt system is predictability. According to Buttazzo [2, p. 10], predictability can only be achieved if the system ensures timeliness, meaning that the value of a process depends not only on its correct output but also on its timely execution—specifically, meeting deadlines.

To fulfill these requirements, the software foundation, particularly the OS kernel, must be tailored to rt needs. Key aspects include preemptiveness, scheduling, inter-task synchronization, inter-task communication, memory management, interrupt handling and timers. The necessity for these mechanisms to be implemented differently in rt capable kernels compared to non-rt ones is discussed in detail in Chapter 2.7.2 and will therefore not be reiterated here.

However, these mechanisms must not only be rt capable at the kernel level—they also need to be considered during the development of applications intended to operate in a rt context. As Buttazzo outlines in [2, p. 13 - 20], particular attention must be paid to the choice of programming language, memory usage and synchronization and communication between processes. It follows that, as shown in Figure 4.2 memory management, inter-task communication and inter-task-synchronsiation are not only requirements for the OS, but must also be taken into account in the test system.

In [2, p. 17, 18] Buttazzo emphasizes that the choice of programming language for rt systems should avoid the use of dynamic data structures and recursion. This is because dynamic memory allocation and deallocation introduce unpredictability in execution time, making it impossible to determine the program's timing behavior in advance. Therefore, memory must be allocated statically and prior to execution. Similarly, recursion poses a challenge, as it is difficult to predict the amount of memory required and the number of recursive calls during runtime.

Furthermore, Buttazzo argues that a programming language suitable for rt applications should support bounded loops, meaning that the number of iterations is fixed and known at compile time. In general, programming languages for rt systems should offer the capability to explicitly define timing constraints and ensure adherence to the aforementioned principles—ideally enforcing them. As programming languages, Buttazzo suggests `Real-Time Concurrent C` and `Real-Time Euclid`, both of which fulfill all the stated requirements.

By contrast, Michael P. Rooney et al. describe in [3, p. 1] a study involving the development of a Supervisory Control and Data Acquisition (SCADA) rt application implemented in the four different programming languages C, Rust, Numba Python and Cython. All of these languages were successfully used to achieve rt performance. Notably, Rust demonstrated a power consumption profile comparable to C, indicating similar efficiency.

While the characteristics outlined by Buttazzo are not mandatory in any of these languages, they can be implemented with appropriate discipline and design. Thus, these languages are also suitable for rt application development, provided that the specific constraints related to memory management, recursion and timing predictability are consciously observed and enforced by the developer, as they are not inherently guaranteed by the language itself.

The chosen programming languages and specific implementation-level requirements for achieving rt capability and how these are addressed are discussed in detail in Chapter 4.4.2.

Furthermore, the test system must be capable of transmitting both rt and non-rt data concurrently over the same network, with the rt traffic maintaining a maximum round-trip delay of 3 ms. This timing constraint stems from the fact that this work seeks to characterize Wi-Fi's rt capabilities and the factors that influence them and to assess whether Wi-Fi can be employed in an industrial rt environment. As discussed in Chapter 2.7, such environments commonly impose a 3 ms deadline. So, beside the evaluation of the rt capabilities of Wi-Fi and the minimization of the Round-Trip-Time (RTT), timing constraint of 3 ms is the goal of this work, like stated in Chapter 1.2.

To summarize the requirements of the test environment are illustrated in Figure 4.2. There the requirements are split between OS and test system. Furthermore, a distinction is made between rt and non-rt requirements. In this section the requirements shown in blue where discussed.

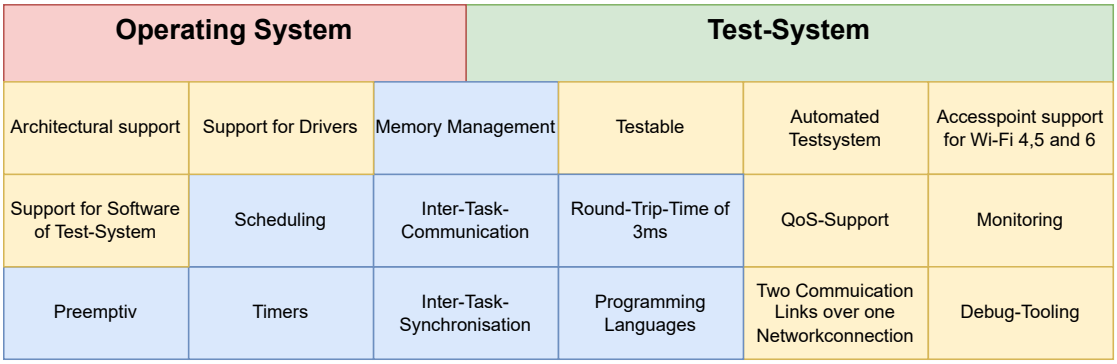

Figure 4.2 Illustration of the requirements for the test environment: Requirements caused by the real-time capability are shown in blue. Requirements related to the test system are shown in yellow.

4.1.2 Requirements of the Test Environment

The following section defines the requirements for the test environment which are shown in yellow in Figure 4.2. For both the test system and the OS, the previously defined criteria for rt capability are assumed to be fulfilled.

The requirements for the OS include providing appropriate support for both the hardware and the software components of the test environment. As described in Chapter 2.1, this work utilizes a Raspberry Pi 5, to which a MediaTek 7925B22M and an Atheros AR5BXB92 Wi-Fi module can be connected via PCIe. Accordingly, the OS have to support the Raspberry Pi 5 architecture and offer compatibility with the drivers mt76, mt7925e and ath9k.

Furthermore, the mt7925e driver is only available starting from Linux kernel version 6.8 and receives extended feature support with newer versions. These enhancements primarily concern functionality related to Wi-Fi 6E and Wi-Fi 7, which is described in [4]. Since these standards are also within the scope of this work, the OS should support the most recent possible version of the mt7925e driver to ensure full access to these capabilities.

In addition to hardware compatibility, the OS must also ensure compatibility with the required software components. This includes the availability of essential programming languages and user-space networking and diagnostic tools which are necessary to run the test-system.

Moreover, the OS should be as minimal as possible, containing only the components explicitly required for the test environment. This reduces background activity and minimizes potential sources of interference or error, thereby ensuring a more stable and predictable testing environment.

In addition to the OS, the test system also has non-real-time requirements. These are highlighted in yellow under "Test System" in Figure 4.2. The application under test must be instrumented so that its rt performance can be measured accurately, enabling meaningful conclusions to be drawn. This requires comprehensive monitoring and debugging facilities to precisely identify and isolate any sources of rt violations.

For the non-real-time traffic, as much data as possible should be transmitted without degrading the rt application's performance. This means, only throughput and packet-loss metrics need be considered for this traffic in the monitoring.

Moreover, this study will compare the rt performance of the algorithms underpinning different Wi-Fi standards. To that end, the server must support access points for Wi-Fi 4, 5 and 6 across all relevant frequency bands (2.4 GHz, 5 GHz and 6 GHz). The impact of QoS on rt capability will also be evaluated, so the test system must be able to mark network packets with QoS parameters. Given the large number of combinations of Wi-Fi standards, frequency bands and QoS settings, an automated test framework is required to generate a comprehensive dataset for all test cases, from which reliable insights can be derived.

In this chapter, the requirements for the test environment were defined. The following sections will discuss how these requirements are implemented. The focus will first be on the selection and configuration of the OS, because it forms the foundation of the test environment after the choice of hardware.

4.2 Operating System

As discussed in the previous chapter, the selection of an appropriate OS must primarily consider its rt capabilities. Such OSs are referred to as Real Time Operating System (RTOS). There exists a wide variety of RTOS, each differing in architecture, strengths and limitations. The following section compares the advantages, disadvantages and performance characteristics of selected RTOSs. Furthermore, their suitability for the purpose of this work is evaluated.

4.2.1 Selection of Operating System

One RTOS that imposes almost no restrictions in terms of performance and compatibility compared to a general-purpose OS is RT-Linux. As described by Sampath et al. in [5], RT-Linux is a Linux-based system enhanced through the PREEMPT_RT patch, which equips it with rt capabilities. The authors examine the use of RT-Linux on an industrial controller with strict rt requirements and additional rt Ethernet demands. Their study demonstrates the suitability of RT-Linux for such applications, reporting a maximum jitter of 380 microseconds over a 24-hour endurance test.

Similarly, Arshiabanu [6] presents a rt system implemented using RT-Linux. The study evaluates the impact of various scheduling policies on rt performance and is executed on a Raspberry Pi. Dewit et al. [7] further investigate the influence of RT-Linux in combination with a Raspberry Pi 5 on scheduling latencies, reporting a maximum scheduling latency of 125 microseconds. This study is especially relevant for this work, because it uses the same hardware platform.

Despite these studies, there also are more critical perspectives. Siro Arthur et al. [8] also evaluated rt capabilities of RT-Linux. This study analyzes metrics such as context switch time, memory access latency and TCP latency. It compares not only various Linux preemption models but also contrasts RT-Linux against RTOS specifically designed for rt use, such as RTAI and LXRT. The study reveals that while the PREEMPT_RT patch enhances rt capabilities, it does not reach the same level of determinism as systems natively designed for rt applications. Furthermore, this analysis was performed on single-core CPUs, where reduced shared resources can change the rt behavior.

There are also studies with multicore platforms which question the suitability of RT-Linux for hard rt environments. Carvalho et al. [9] investigated RT-Linux running on a Raspberry Pi, focusing on latency metrics such as polling, timed loops, hardware interrupts and software interrupts. Their results confirm that although RT-Linux offers certain rt properties, it should only be used in non-critical rt environments. This conclusion is supported by Betz et al. [10], who state that RT-Linux is appropriate for soft rt applications.

Yang et al. [11] compare the rt performance of NuttX, FreeRTOS, RT-Linux and Xenomai. A noteworthy aspect of their study is the inclusion of both rt capabilities and general-purpose features. While RT-Linux can be used as a standalone system, to match these requirements, pure RTOS like FreeRTOS and NuttX are executed alongside a general-purpose OS. Also in this study a Raspberry Pi 4 was used as the hardware platform. The results are showin that pure RTOS, especially FreeRTOS, netherless deliver better rt performance.

Mariesk et al. [12] distinguish between the two fundamental RTOS types kernel-based RTOS and embedded RTOS. In their study, they evaluate systems based on interrupt latency, task switching time and preemption time. RT-Linux and Xenomai are considered kernel-based RTOS, while eCos represents the embedded type. The results show that kernel-based RTOS are more suitable for multi-task applications due to their lower task-switching overhead. Since this work requires the execution of multiple concurrent tasks, frequent scheduling switches are expected, making a kernel-based RTOS the preferable choice.

In summary, the optimal RTOS depends heavily on the specific application and rt requirements. Based on the referenced studies, FreeRTOS appears to offer the best rt capabilities. However, additional constraints beyond rt performance must be considered when selecting the OS, like stated in the previous section. This means particularly hardware compatibility of the OS. The MediaTek 7925B22M Wi-Fi card used in this work lacks drivers outside the Linux ecosystem, thereby excluding FreeRTOS as a viable option.

Considering the experience with RT-Linux on Raspberry Pi systems stated by the studies before and the demonstrated maximum scheduling latency of 125 microseconds along with a maximum jitter of 380 microseconds of the test-system, RT-Linux proves sufficient for the purposes of this work. Given a round-trip time (RTT) requirement of 3 milliseconds, this jitter is acceptable. Furthermore, since Wi-Fi is inherently not rt capable and as discussed in Chapter 2.1, the employed hardware which has, among other shared resources, a split L3 cache, is not optimized for rt processing. So the minor limitations of RT-Linux do not significantly impact this test-environment.

There are various ways to implement an RT-Linux system. As described by Rostedt in [13], a RT-Linux system can be created by extending the kernel with the $PREEMPT\_RT$ patch. This results in two primary approaches for building an RT-Linux system. One can either use an existing Linux distribution such as Ubuntu and apply the $PREEMPT\_RT$ patch, or build a complete Linux system from scratch using a build system.

As Vasquez outlines in [14, Chap. 6], building an OS using a build system offers the advantage of full customization tailored to specific needs. Every component of the OS can be configured and assembled individually. However, this method is considerably more time-consuming compared to using a pre-built OS that is already available for the Raspberry Pi 5, such as a customized Ubuntu version 24.02 or Raspberry Pi OS, which can then be extended with the $PREEMPT\_RT$ patch.

Due to the requirements defined in Chapter 4.1.2, the OS must be capable of providing a foundation for both software and hardware. For the Wi-Fi card drivers, it is essential, as previously mentioned, that the kernel is as recent as possible. The kernel version can only be adapted with a reasonable effort, in the existing Operating System for the Raspberry Pi 5. A build system has a possibility, to do this more easily. Moreover, certain kernel configurations must be set to support integration of these drivers. These configurations can be managed much more flexibly and conveniently using a build system. For this reason, the OS used in this work

is built using a build system.

Furthermore, an OS built using a build system is the preferred choice for embedded systems, as noted by Navik in [15]. This assertion is supported by Vasquez in [14, Chap. 6], who argues that the main advantages lie in the minimal footprint and the ability to restrict applications and background processes as needed. Given that the requirements for the embedded system in this work also include reproducibility and minimal external interference, this further supports the decision to use a build system for the OS.

As Salvador describes in [16, Chap. 1], the Yocto Project is a widely used and powerful build system. It is open-source and comes with excellent documentation, making it a suitable choice for building the OS in this work.

The technical details of how the Yocto Project functions were already covered in Chapter 2.2. Since all technologies used in the test system must be known for proper OS configuration, the following chapter will provide a more detailed overview of the test setup.

4.2.2 Structure of Test Environment

Figure 4.3 illustrates the test environment along with all relevant components and technologies. As previously described, it consists of a server and a client. First, the structure of the test system is explained in more detail, followed by an analysis of how it satisfies the requirements defined in Section 4.1.2.

The test environment in Figure 4.3 is divided into several elements. The OS is excluded from the discussion and the focus is placed solely on the components of the test system.

The server part of the test system is controlled via the **Server Test Suite**, which is responsible for executing test runs and automating the overall testing process. For each combination of access point configuration and QoS level, the test suite performs a complete test run. Therefore, the total number of test runs is defined as $N_T = N_{AP} \times N_{QoS}$, where N_T is the total number of test runs, N_{AP} is the number of different access point configurations and N_{QoS} is the number of different considered QoS levels. The Server Test Suite is essential for the automated execution of these test runs, thereby fulfilling the previously specified requirement for test automation.

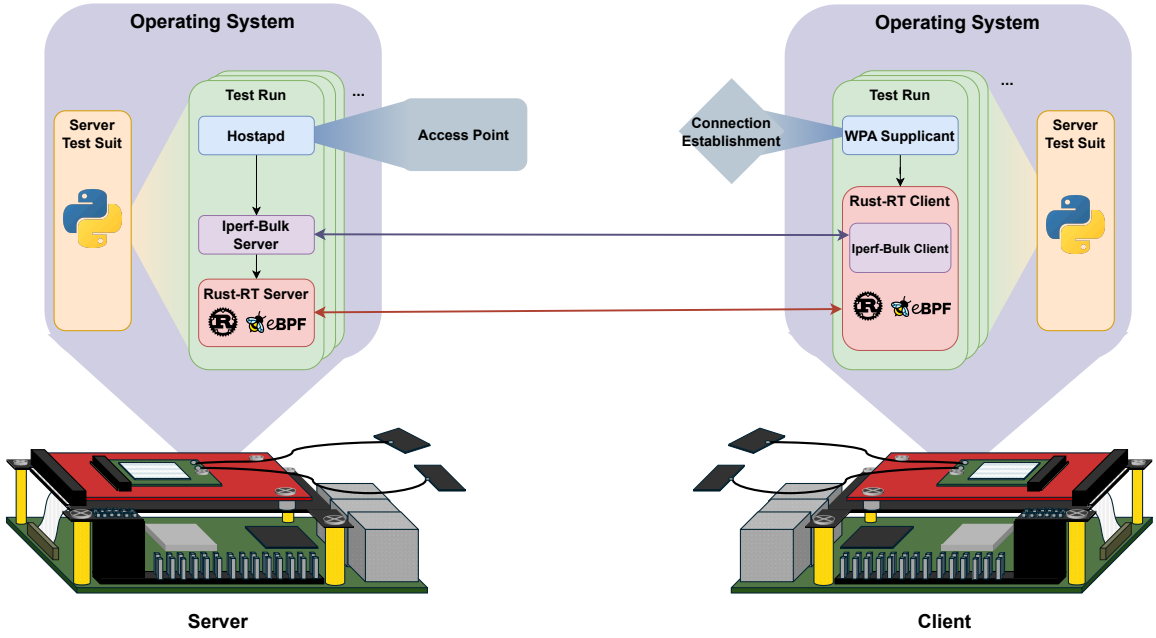

Figure 4.3 Illustration of the test environment including used technologies.

A **server-side test run** comprises the three components Hostapd, Iperf Bulk Server and

Rust-RT Server. Each test run contributes to the overall test automation by ensuring that the required steps to establish both a rt and a non-rt network link within the same network are executed in the correct order.

The first step in a server-side test run is the creation of the Access Point (AP), which is achieved using the **Hostapd** software. Hostapd allows the configuration of various Wi-Fi standards, frequencies and bandwidths, thereby meeting the requirement to support multiple Wi-Fi standards.

Once the AP is established, the server for non-rt applications is launched using **Iperf3**. This Software supports differentiation between UDP and TCP traffic, various packet sizes and data rates. It also provides metrics such as packet loss and jitter, thus fulfilling all previously defined requirements for non-RT data transmission.

Finally, the **Rust-RT Server** is started. This component is implemented in Rust and makes use of Berkley Package Filter (BPF). This enables both compliance with rt requirements and integrated monitoring capabilities. The rationale behind the choice of programming languages is discussed after the explanation of the test system architecture in Section 4.2.3.

The **client-side test system** is structurally similar to the server-side system. As shown in Figure 4.3, it also consists of a Client Test Suite and a test run. The functionality of the client-side test suite and test run mirrors that of the server-side system. However, the client-side test run differs slightly in structure.

The client-side test run comprises two phases. First, a connection to the server's AP must be established. This is handled using the *wpa_supplicant* software, which utilizes a configuration file specifying details such as the network password and encryption algorithm. For this reason, client and server test runs must be synchronized. Any mismatch in Wi-Fi standards will prevent the client from connecting to the server's AP.

Once a connection is established, the **Rust-RT Client** is launched. This client establishes a connection to the Rust-RT Server. Unlike the server-side test run, the Iperf client is not launched directly as part of the test run but is instead initiated from within the Rust-RT Client. The reason for this design choice is explained in more detail in Section 4.4.2.

After describing the architecture and functionality of the test system and how it satisfies the specified requirements, the following section discusses the selection of technologies and their configuration within the Yocto environment.

4.2.3 Used Technologies

To create an access point on the server, the software `hostapd` is used, as previously described. As Anderson states in [17], this is one of the most widely used tools to create an AP that can be configured in detail. With this software, the Wi-Fi standard, frequency, bandwidth and encryption can be configured, as shown in [18]. Thus, all previously specified requirements regarding the configurable properties of the Wi-Fi are met by this software.

To allow the client to connect to the AP, the software `wpa_supplicant` is used. As shown in [18] and [19], it can be configured at the same level as `hostapd`. This allows for a fine-grained configuration of the connection to the AP. An alternative on Linux would be the NetworkManager. However, it includes more background processes and is therefore more heavyweight, which would impact the system more strongly. Moreover, it cannot be configured as precisely in coordination with `hostapd` as `wpa_supplicant`.

To generate network traffic, `iperf3` is used, as it fulfills the required specifications, as previously described. As shown in [20], this tool allows the generation of network traffic and precise configuration regarding TCP/UDP, packet size and throughput. Furthermore, no separate monitoring needs to be developed, as the tool already includes it. For example, latency and packet loss can be directly observed using `iperf3`.

Both the server test suite and the client test suite are implemented in Python. This is possible because there are no real-time requirements for these components. Only the server responsible

for the RT connection must fulfill real-time constraints. In this case, Python is a suitable choice because, as described in [21], it is very popular and widely used in embedded programming. Moreover, the language offers a high level of abstraction and is script-like in nature. Since the test suites only need to coordinate the testing processes, the script-based nature of Python is well-suited for this task.

In contrast to the test suite, the RT server and client must fulfill real-time characteristics. As described by DLR in [22, p. 86], most embedded real-time applications are written in C or C++. Nevertheless, Rust was chosen for the RT application in this work. This decision is based on the fact that DLR also states in [22] that Rust is more suitable for RT applications due to its memory and thread safety. However, the language is still too young to be fully established for hard real-time systems. Rust does not use a garbage collector, as explained in [23, p. 2]. Nevertheless, it prevents issues such as use-after-free, null pointer dereferencing and data races. This is achieved through Rust's ownership system. Ownership in Rust means that each value has exactly one owner, which becomes invalid once the owner goes out of scope. An owner can be a variable, function, or data structure that holds the exclusive right to access or modify a value. This mechanism also prevents undefined behavior. Furthermore, as described in [23, p. 1], Rust offers better development capabilities due to `rustc` and zero-cost abstractions. `rustc` allows better debugging and significantly eases troubleshooting. Rust provides better error messages than C and benefits from zero-cost abstractions. Additionally, as shown in [3, p. 1], Rust performs similarly to C. Rust is also becoming increasingly popular, as noted in [22, p. 2]. This is also evident in its adoption into the Linux kernel due to its strong memory safety. As described by Panter in [24], this integration has already shown promising results. Rust has also been integrated into Android for memory safety reasons. As described in [25, p. 1], the share of safety vulnerabilities was reduced from 76% to 35%. Moreover, increasing progress is being made in the field of real-time applications with Rust. As shown in [26], the Safety-Critical Rust Consortium was founded in 2024 with the goal of using Rust in DO-178C-certified products. As described in [27] and [25, p. 4], this is one of the strictest certifications for airborne systems. In summary, Rust is well-suited for real-time applications and offers better development capabilities than C and C++. It is also becoming increasingly popular and has almost no performance disadvantages compared to C. Accordingly, the RT application in this work is written in Rust.

To implement monitoring of the real-time application, BPF is used. It can be seamlessly integrated into Rust code using `libbpf-rs`. BPF, is a technology for efficient in-kernel data processing and monitoring, originally designed for packet filtering. As described by Gregg in [28, Chap. 2.10], it was originally build to observ network packages. And it offers nearly the same functionality as the widely used profiling tool `perf`. Furthermore, it is also employed in practice for network monitoring. This is evident in [29] and [30]. However, it integrates better with Rust and is more flexible due to its scriptability and access to a broader range of event sources than `perf`. BPF supports more event sources such as tracepoints, kprobes, uprobes and performance counters, enabling more detailed and application-specific monitoring.

In summary, the programming languages Python, Rust and BPF, as well as the tools `hostapd`, `iperf3` and `wpa_supplicant`, must be provided by the operating system for the test system. The following section describes how this can be realized using the Yocto Project.

4.2.4 Implementation

The background and functionality of the Yocto Project have already been described in Section 2.2. Therefore, this section does not elaborate on them further and proceeds directly with the specific implementation within the context of this thesis.

Figure 4.4 illustrates the structure of the Yocto Project as used in this work. The figure displays the various layers of the Yocto Project that were utilized. Layers that required no modifications are marked in green. Layers that were integrated but still required certain adjustments are highlighted in yellow. The red-colored `meta-masterthesis` layer was entirely developed as

part of this thesis and contains all necessary customizations to the base image in order to meet the project-specific requirements.

The Yocto Project used in this thesis incorporates the layers `poky`, `meta-openembedded`, `meta-raspberrypi` and `meta-rust`. The `poky` and `meta-openembedded` layers serve as the foundational layers, as is common in most Yocto-based projects, as discussed in Chapter 2.2.

From the `meta-openembedded` layer, the sublayers `meta-oe`, `meta-python` and `meta-networking` are utilized. The `meta-python` layer provides support for the Python programming language, which is required by some of the tools used in the testing environment. The `meta-networking` layer supplies essential networking tools that are also necessary for the test setup.

The `meta-raspberrypi` layer is essential for hardware-specific support for the Raspberry Pi platform. This layer enables the generation of a customized operating system image that is compatible with the Raspberry Pi 5, by providing the required kernel configurations, bootloader integration and board-specific device trees.

The `meta-rust` layer is included to integrate the Rust programming language into the Yocto build system. This integration is necessary to develop the rt application in Rust, as required by the objectives of this thesis.

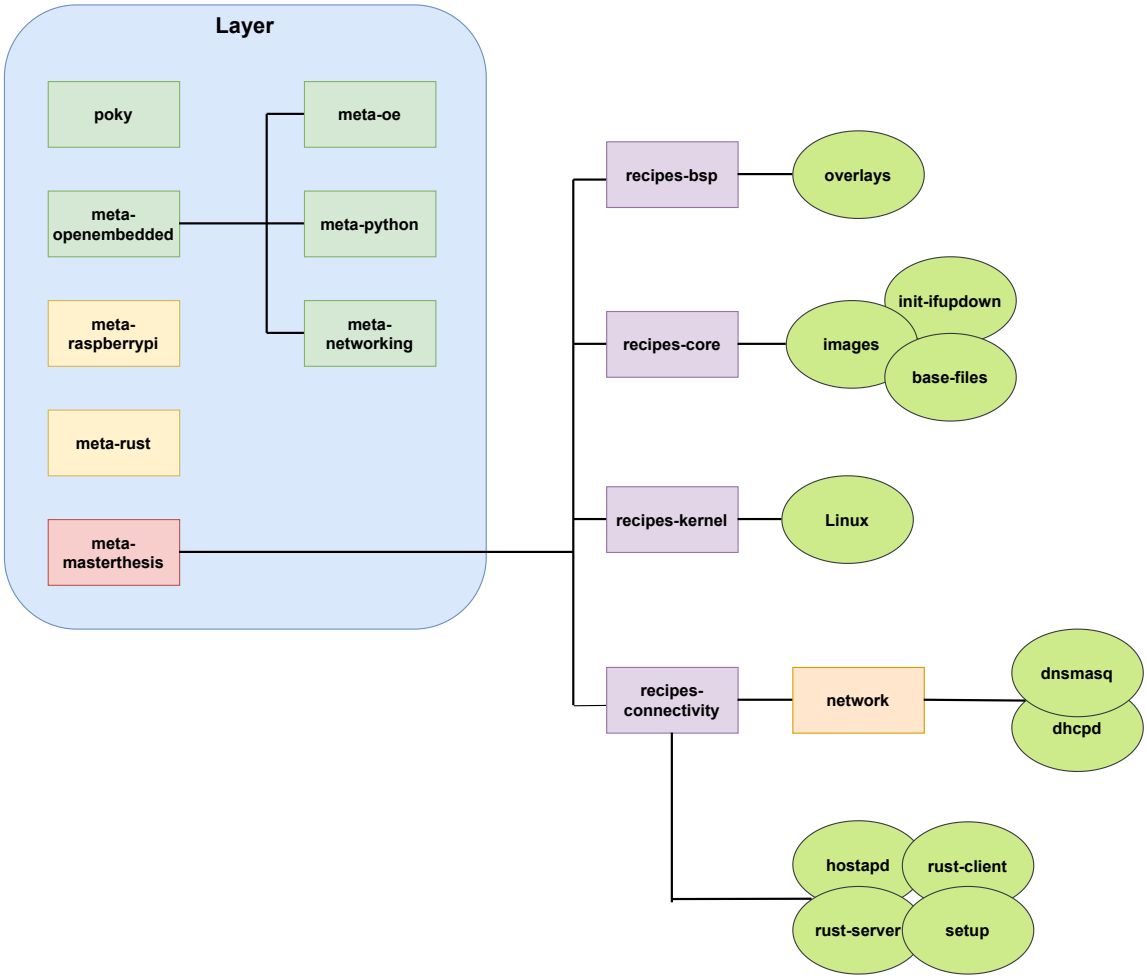

Figure 4.4 Representation of the Yocto Project structure of this work: Layers that did not need to be edited are shown in green, layers that needed minor editing are shown in yellow and custom layers are shown in red.

All layers are built using Yocto version 5.2.1. This corresponds to the `walnascar` branch. As of in the mid of June, this was, besides the `master` branch, the most recent branch of the Yocto Project, as shown in [31]. The latest branch was selected to take advantage of the most up-to-date kernel version included therein.

The kernel version is crucial for the support of the `mt7925e` driver, which is only available starting from kernel version 6.8 and is better supported in even newer kernel releases, as described in [4]. The `walnascar` branch was available for all layers except for the `meta-rust` layer. In this case, it was necessary to manually add the `walnascar` branch to the `layer.conf` file.

Due to compatibility issues, the variable `RUST_LIBC` in this layer had to be replaced with `${@d.getVar('RUST_LIBC').lower()}`. This variable also needed to be defined in the `local.conf` file using `RUST_LIBC = "gnu"`. The GNU C Library was chosen because it is the default C library used in full-featured distributions for the Raspberry Pi.

`local.conf` must also define the target machine to be the Raspberry Pi 5. This ensures proper activation of the `meta-raspberrypi` layer. Furthermore, the preferred Linux kernel version must be explicitly set to 6.12, since the default version provided by the `meta-raspberrypi` layer is 6.1.

The custom `meta-masterthesis` layer is assigned a higher priority than the other layers in `bblayers.conf`. This is necessary to ensure that the `.bbappend` files within this layer are processed before the corresponding `.bb` files of other layers. The `meta-masterthesis` layer can be divided into four recipe categories, illustrated in purple in Figure 4.4. The individual `.bb` and `.bbappend` files are depicted in light green in the same figure.

The Wi-Fi cards used in this work utilize 32-bit DMA addressing. The Raspberry Pi 5 does not support this addressing mode by default, as it uses 64-bit DMA addressing. Therefore, a device tree overlay must be used to enable 32-bit DMA support. A custom recipe was created to install the overlay `pcie-32bit-dma-pi5.dtbo`. This overlay can be obtained from the official Raspberry Pi firmware repository [32]. The recipe installs the locally available overlay into the `/boot/overlays` directory on the target system.

By setting the variables `RPI_USE_OVERLAYS` and `RPI_EXTRA_CONFIG` in `local.conf`, the overlay can be included in the `config.txt` file. To fully integrate the overlay into the operating system, the `meta-raspberrypi` layer must be modified. Specifically, the overlay must be added to the `RPI_KERNEL_DEVICETREE_OVERLAYS` section of the file `meta-raspberrypi/conf/machine/include/rpi-base.inc`. After this adjustment, the overlay was successfully deployed and referenced in the `config.txt` of the generated image.

To make the operating system real-time capable, the Linux kernel must be extended using the RT patch. This is handled within the `recipes-kernel` section of the `meta-masterthesis` layer. The Linux recipe provided by the `meta-raspberrypi` layer, which is based on version 6.12.1, is extended via a `.bbappend` file. However, the RT patch is officially only available for kernel version 6.12.28. Since this version is not compatible with 6.12.1, an older RT patch developed for version 6.12.8 was tested and found to work reliably. This patch was applied by integrating it into the Linux recipe using the `.bbappend` mechanism.

It is also essential to modify the kernel configuration to select the appropriate preemption model. All necessary kernel configuration options can be integrated by placing a `.cfg` file into the source directory of the Linux recipe. This ensures that the final kernel included in the image is real-time capable.

The `recipes-core` category includes extensions to the `init-ifupdown` and `base-files` recipes via `.bbappend` files. The `init-ifupdown` extension introduces a custom `interfaces` file to assign static IP addresses to specific network interfaces. The `base-files` extension defines environment variables and configs that are automatically set upon opening a shell session. This is accomplished by writing shell commands into `/etc/profile.d/myenv.sh`.

The main component of `recipes-core` is the custom image recipe. This extends `core-image` to include all necessary technologies for the test system. It includes self-developed `.bb` files as well as required tools and programming languages.

Once all technologies are installed on the target system, the final step is to integrate the application code and network configurations. This is done in the `recipes-connectivity` section. In the `network` category, marked in orange in Figure 4.4, two `.bbappend` files customize the configurations of `dnsmasq` and `dhcpcd`.

Other `.bb` and `.bbappend` files in this section install applications from a custom Git repositorie onto the target system. The `hostapd.bbappend` file installs multiple AP configurations into `/etc/hostapd/`. Additionally, it modifies the original recipe to compile `hostapd` using a custom configuration that supports the Wi-Fi 6E standard. By default, `hostapd` does not support access points operating in the 6 GHz band.

The `setup.bb` file installs Python-based test suites from a GitHub repository. As these applications are written in Python, no prior compilation is required.

The `rust-client` and `rust-server` recipes were generated using the `cargo-bitbake` tool. This tool generates Yocto-compatible recipes for Rust projects, as explained in [33]. It requires a Git repository reference in the `Cargo.toml` file and all dependencies to be vendored locally. The `cargo vendor` command, as described in [34], is used for this purpose.

The generated recipes were extended to install not only the compiled binaries but also the full source code on the target system. Additionally, the recipe must include checksums for the vendored files.

To enable eBPF integration in Rust, the file `vmlinux.h` is required. This file contains auto-generated headers that are used by the BPF program. To generate it, the kernel configuration option `DEBUG_INFO_BTF=y` must be enabled. The file can then be created using the command: `$BPFT btf dump file "${VMLINUX_PATH}/vmlinux" format c > ${S}/src/bpf/vmlinux.h`.

With these extensions, the Rust applications—including binaries, source code and a local compilation environment—are fully installed on the target system.

In summary, this chapter described the steps required to build an operating system using the Yocto Project that fulfills all requirements for the intended test environment. To explain the implementation of the test system an explanation of monitoring in general and the functionallity of the used technologies in monitoring is necessary. So this will be described in the following chapter.

4.3 Monitoring

As already mentioned in Chapter 4.2.3, BPF is used for monitoring, which is integrated into the Rust code via `libbpf-rs`. The following section explains what BPF is, how it works, and how it is used to implement the monitoring of the test system. The integration into Rust is described in more detail in Chapter 4.4.2.

4.3.1 Berkley Package Filter

BPF was originally developed in 1992 by McCanne and Jacobson, as presented at the USENIX Winter Conference 1993 [35]. It was significantly extended in 2013 by Alexei Starovoitov, laying the foundation for extended Berkley Package Filter (eBPF), as indicated in this patch thread [36]. eBPF is the modern standard for BPF and is also used in this work. However, the term BPF will be used throughout this document, as it is common in the literature and sources, as described by Gregg in [28, Chap. 1]. Brendan Gregg has played a significant role in the dissemination and development of BPF, as evidenced by his books [28] and [37].

As Gregg explains in [28], writing BPF code directly is very challenging. Therefore, various libraries exist to simplify this process. The two most prominent are `bcc` and `libbpf` for C++ and C, respectively. Using the tools `bcc` and `bpftrace`, BPF can be utilized from the command line. These tools also internally rely on `libbpf` and `bcc`.

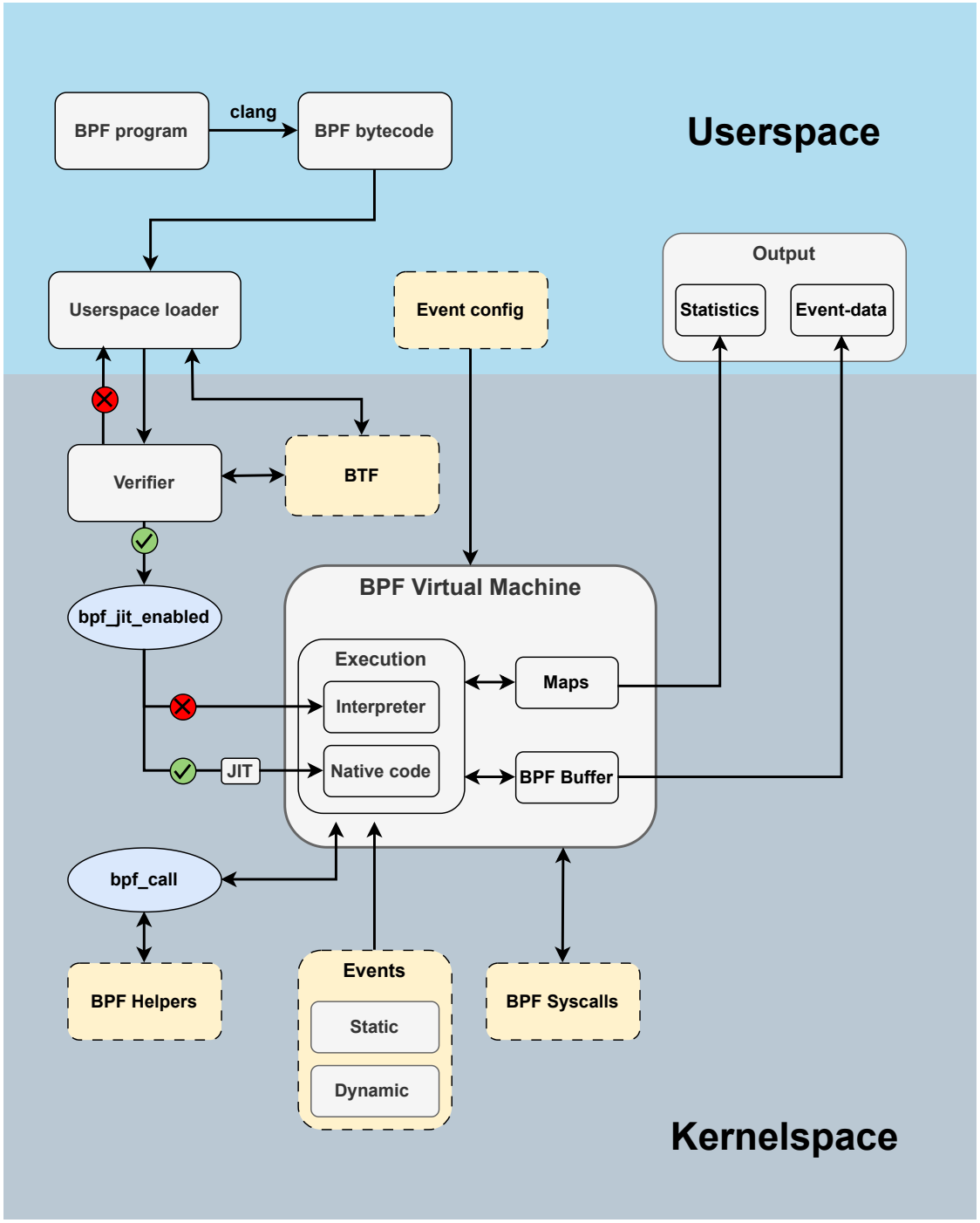

Figure 4.5 Presentation of the working principle of Berkley Package Filter

In this work, only the `libbpf-rs` library is used, and thus, the functionality and scope of these tools will not be further discussed. The `libbpf-rs` library is based on the `libbpf`

library and serves as its Rust binding. Its exact functionality will be described in Section 4.4.2, as it is closely related to the implementation of the server and client. The following section provides a more detailed explanation of the functionality of BPF.

In Figure 4.5, the working principle of BPF is illustrated. The figure shows the various components of BPF and their interactions. It also distinguishes where each component resides, either in the kernel or in user space. In Figure 4.5, components that are directly connected to the BPF workflow are depicted in grey. Configurations or interactions are represented in blue circles, while external Linux components are shown in yellow.

The BPF workflow begins with a BPF program. As explained by Gbadamosi et al. in [38, p. 7], this program is compiled using `clang`. For this compilation, the `bpf` target must be set. This process produces the BPF bytecode, which is then loaded into kernel space via a userspace loader, such as one of the available libraries like `libbpf`, to the verifier. As described by Gbadamosi et al. in [38, p. 4], the userspace loader employs the `BPF_PROG_LOAD` system call. Additionally, the necessary kernel hooks and maps are created and linked to the BPF program.

The verifier checks that the program does not perform any illegal operations and that the integrity and safety of the kernel are not compromised. As Gregg explains in [28, Chap. 2.3.2], this verification not only detects disallowed instructions but also identifies bugs or other potential issues that could trigger a kernel panic or create security vulnerabilities. Unbounded loops are also disallowed, as a BPF program must always terminate within a finite amount of time. The verifier relies on BPF Type Format (BTF) to ensure that the code is functional.

As described by Gregg in [28, Chap. 2.3.9], BTF is generated by the Linux kernel and stored in the file `vmlinux.h`. In this work, BTF was already generated via Yocto, as described in Section 2.2. BTF contains metadata and structure definitions for Linux kernel headers and all kernel `struct` types. Normally, this information is included with the Linux headers. However, in many cases, the `struct` definitions in the headers are incomplete or missing. As a result, a BPF program that, for example, relies on a syscall whose return value is an undefined `struct`, would be rejected by the verifier.

Beyond verification, BTF also plays an important role in providing debugging information for BPF programs. It is a compact, binary-encoded representation of type information, designed to be efficiently processed by both the kernel and user space tools. With BTF, tools such as `bpftool` can inspect kernel data structures, and debuggers can provide symbolic names instead of raw memory addresses or offsets. BTF forms the foundation for Compile Once - Run Everywhere (CO-RE). In order for BPF code to run anywhere after compilation, the used `struct` definitions must be precisely described in the BTF. This information must be passed to both the userspace loader and the verifier. Since CO-RE is not used in this work, it will not be described further here.

If the program is accepted by the verifier, it is either compiled by the Just-In-Time (JIT) compiler or executed by the interpreter. For the JIT compiler to take over, it must be both supported and enabled, which is controlled via the `bpf_jit_enabled` operation. As described in [39], this option can be enabled by writing the value "1" to the file `/proc/sys/net/core/bpf_jit_enable`.

Like described by Gbadamosi et al. in [38, p. 4] the JIT compiler translates the BPF bytecode into native machine code for the running architecture at load time. This eliminates the overhead of interpreting instructions during execution and significantly improves performance, especially for programs that are invoked frequently or handle large volumes of events.

If the JIT compiler is not enabled or supported, the BPF program is executed using the in-kernel BPF interpreter. The interpreter processes each BPF instruction sequentially, decoding and executing them in a virtualized manner. While this approach is slower due to per-instruction overhead, it ensures compatibility across architectures and provides a fallback execution path in environments where JIT compilation is unavailable or undesired.

Van Geffen describes in [40] a custom JIT compiler that offers greater security than the Linux implementation. According to [40, p. 566], this compiler achieves a performance improvement

of 5.24× over the interpreter. However, it remains 1.82× slower than the Linux JIT compiler. From these results, the performance gain of the Linux JIT compiler over the interpreter can be calculated as $5.24 \times 1.82 = 9.53$. Since this difference is close to a factor of ten, and the monitoring overhead on the test system should be minimized, this work employs the Linux JIT compiler.

As described by Gregg in [28, Chap. 2.3], once the BPF code has been verified and translated into native instructions, it is executed within a virtual machine running in kernel space. Gregg notes in [28, Chap. 2.3] that, particularly after JIT compilation, this virtual machine exists only as a specification. In contrast to the definition of a virtual machine provided by Bugnion in [41], where a virtual machine represents an abstraction of an entire computer system, the BPF virtual machine is purely a specified execution model. According to Bugnion, a true virtual machine must virtualize the processor, memory, and I/O components. In the context of BPF, as Fleming explains in [42], the term "virtual machine" refers solely to a logical Central Processing Unit (CPU) specification with a fixed instruction set and ten registers, designed to run platform-independently within the kernel. Consequently, as Gregg observes, the BPF virtual machine exists only in specification form. This is particularly true when the code has already been compiled into machine instructions. In such cases, these instructions are executed directly by the CPU. During the runtime of a BPF program, there is no additional abstraction layer between the CPU and the executing code. A JIT compiler and an interpreter exist, but there is no virtual machine in the traditional sense.

Besides the 10 registers, the virtual machine also offers 512 bytes on the stack, as Gregg describes in [28, Chap. 2.3].

Additionally, more memory is available in so-called maps. These maps are created using the command `BPF_MAP_CREATE`, as described in [38, p. 5].

As Gregg discusses in [28, Chap. 2.2], BPF programs can exchange data among each other threw using maps. Furthermore, the maps are synchronized, allowing parallel memory accesses. Maps can be accessed from userspace, providing a way to transfer collected data from the virtual machine back to userspace. Command-line tools such as `bpftrace` use these maps to display statistics, for example.

As Gregg explains in [28, Chap. 2.2], this has the advantage that no communication between user- and kernelspace is necessary. Since the program runs entirely in kernelspace at runtime, different event sources can be filtered and processed there. Event sources or network packets can also be discarded in kernelspace if they are harmful or would endanger further processing in userspace due to their value ranges. Thus, the mechanism of maps and the virtual machine specified therein ensure security and efficiency by preventing unnecessary kernel-to-userspace interactions.

Gregg uses in [28, Chap. 2.6] the event source as an umbrella term for events and probes, so for static and dynamic event sources. Since this term is inconvenient, in literature often the term static and dynamic envents is used, or in general it is not used uniform. So to be more precise in the following the term event is only used for static event sources and the term probes is only used for dynamic event sources.

As shown in Figure 4.5, the event configuration from userspace interacts with the virtual machine in kernelspace. The event configuration particularly refers to the settings of command-line tools. Among other things, it can be configured which event sources should be tracked, for how long they should be tracked, and how they should be processed. Since command-line tools are not used in this work, the event configuration will not be discussed further here, but it is mentioned for completeness and illustrated in the figure.

As shown in Figure 4.5, the virtual machine provides another method for transferring data from the kernel to userspace.

This is the ring buffer, which Gregg in [28, Chap. 2.3] also describes as the perf buffer. The buffer is used to continuously transfer data from the kernel to userspace. There are also other

variants for sharing data between user and kernel space, including relayfs.

RelayFS was developed by Tom Zanussi and has been integrated into the Linux kernel since 2003, as evident from the patch thread in [43]. As explained by Zanussi in [44], it is also a mechanism to efficiently transfer data from kernelspace to userspace. In this approach, a ring-shaped buffer is allocated per CPU, which can be read from userspace using `mmap()`. This minimizes copy operations because both the kernel and userspace access the same memory region directly without intermediate data duplication. Writing to these buffers is performed sequentially by the kernel, while the userspace continuously reads and only updates the read pointers. As described in [44, p. 498], it was originally introduced as a standalone filesystem under `/mnt/relay`. It is now integrated into the tracing subsystem and is used, among other things, by the kernel tracing tool LTTng, as described in [45, p. 89].

Although RelayFS is older than perf and ring buffers, it still has use cases, as Fournier describes in [45]. Gregg and Calacera describe the functionality and usage of BPF in great detail in their books [28] [46]. However, neither of them mentions any method other than perf buffer and maps for exchanging data between user and kernel space. This tendency is reflected not only in theory but also in practice.

Nair et al. developed GAPP in [47], a tool to identify serialization bottlenecks in parallel Linux applications. For monitoring, BPF is also used, and maps and perf buffers are employed for data exchange between kernel and userspace. Another example is Liu et al., who built a protocol-independent network observability system in [30]. Furthermore, both Kannan et al., presenting a network monitoring agent in [29], and Gowda et al., proposing kernel-level request tracing for microservice visibility in [48], use BPF along with maps and perf buffers.

All these works utilize BPF for monitoring and rely on maps and perf buffers for communication between kernel and userspace. These examples confirm, on one hand, that BPF is widely used in network observability. On the other hand, it is evident that no alternative methods for kernel-user communication are employed in practice. This is likely because maps and perf buffers are well integrated with BPF and offer excellent performance. As a result, the effort to integrate, for example, RelayFS with BPF is not justified. Therefore, this work also relies on maps and perf buffers.

The modules not yet mentioned from Figure 4.5 are BPF helpers, event sources, and BPF syscalls. All of these are shown as yellow boxes, which, as explained earlier, indicates that they represent components, which do not belong fully to BPF, but influence the execution of the BPF program.

As described by Gregg in [28, Chap. 2.3.6], a BPF program cannot execute arbitrary kernel functions. To enable access to information from the operating system context within the program, BPF helper functions are used. These are defined in the BPF API in `include/uapi/linux/bpf.h`, and new helpers are constantly being added. The BPF API in this context refers to the set of functions and interfaces that the kernel exposes to BPF programs to interact with kernel and user space resources in a controlled manner. As illustrated in Figure 4.5, these helpers can be invoked via the `bpf_call` instruction.

The following paragraphs present selected helper functions to provide an overview of their capabilities. Among these restricted kernel calls is `bpf_ktime_get_ns()`, which returns the time since kernel start in nanoseconds. This enables access to a monotonic clock, allowing the measurement of time intervals between specific event sources at kernel level.

Other important BPF helpers are `bpf_get_current_pid_tgid()` and `bpf_get_current_comm(buf, buf_size)`. Each of these functions has a return value, which will be referred to as R_1 and R_2 in the following. R_1 returns both the TGID and the PID of the current process, where TGID (Thread Group ID) refers to the identifier of the process as a whole, and PID (Process ID) refers to the specific thread within that process. The 64-bit value returned by R_1 is composed such that the high 32 bits contain the TGID and the low 32 bits contain the PID. R_2 returns the current task name, also known as the command name (`comm`)

of the process. With these two functions, process names and precise IDs can be determined both in user space and kernel space. This is important for filtering, and also for diagnostics when it is necessary to determine which process triggered a given event source.

Another essential kernel function accessible via helpers is `bpf_trace_printk`. As described in [49], this function can write print messages to the kernel trace log. It thus serves as the `printf` equivalent for BPF. In contrast to C, where, as explained in [50], messages are printed to the console, `bpf_trace_printk` writes them to `/sys/kernel/tracing/trace`, because BPF applications have no console available. The messages can then be read via the `/sys/kernel/tracing/trace_pipe` interface. This function therefore provides a fast way to add simple debugging output.

As illustrated in Figure 4.5, beside BPF helper funtions, also syscalls can be invoked from the BPF virtual machine. These syscalls are specific to the BPF environment and provide the same functionality as conventional syscalls, but operate differently.

Syscalls have existed since the early 1980s, as evidenced by the first official Unix Programmer's Manual [51]. As described by Tanenbaum in [52, p. 50–53], syscalls constitute the programming interface for userspace programs to request services from the kernelspace. Thus, syscalls act as a mechanism for the userspace to interact with the kernelspace.

According to Tanenbaum [52, p. 54–55], syscalls can be divided into the four fundamental categories process management, file management, directory and file system management, and miscellaneous. System calls for process management include, among others, the `fork()` command, which creates a child process. The `open()` and `close()` system calls are used for file management to open or close a file. The `mkdir()` command creates a new directory and falls into the category of directory and file system management. The miscellaneous category contains everything that does not fit into the three categories mentioned above, such as the `chmod` system call, which modifies a file's permissions by changing its protection bits.

As described by Gregg in [28, Chap. 2.3.6], the BPF syscall can be invoked via the code shown in listing 1.

```
int bpf(int cmd, union bpf_attr *attr, unsigned int size);
```

Listing 1 BPF syscall interface

The `cmd` parameter specifies the particular BPF operation to be performed. It is an integer value provided through an enumeration. As Gregg describes it in [28, Chap. 2.3.6], the `BPF_PROG_LOAD` enum value instructs the verifier to check a BPF program and subsequently load it into the kernel. Similarly, maps can be accessed or modified from userspace using commands such as `BPF_MAP_CREATE` or `BPF_MAP_LOOKUP_ELEM`.

The key difference between conventional syscalls and the BPF syscall is that there is only one syscall for the entire BPF subsystem. Nevertheless, as explained above, this single syscall can provide multiple functionalities. In contrast, conventional syscalls are monofunctional, with one syscall per functionality. Thus, while traditional syscalls are monofunctional, the `bpf()` syscall is polyfunctional. Compared to traditional syscalls, `bpf()` does not directly implement a single syscall in the classical sense, but rather acts as a wrapper for the syscalls of the BPF subsystem. However, the fundamental purpose of syscalls, to provide a communication interface between userspace and kernelspace, remains unchanged in the BPF implementation.

As shown in Figure 4.5, event sources also interact with BPF. As described by Gregg in [28, Chap. 1], the primary functionality of BPF is to execute user-defined code in response to event sources. Therefore, the event sources shown in Figure 4.5 are among the most crucial components of the overall system. In Figure 4.6, a software stack is illustrated. This illustration indicates, among other aspects, where specific static and dynamic event sources are located and which monitoring techniques can be used to observe them.

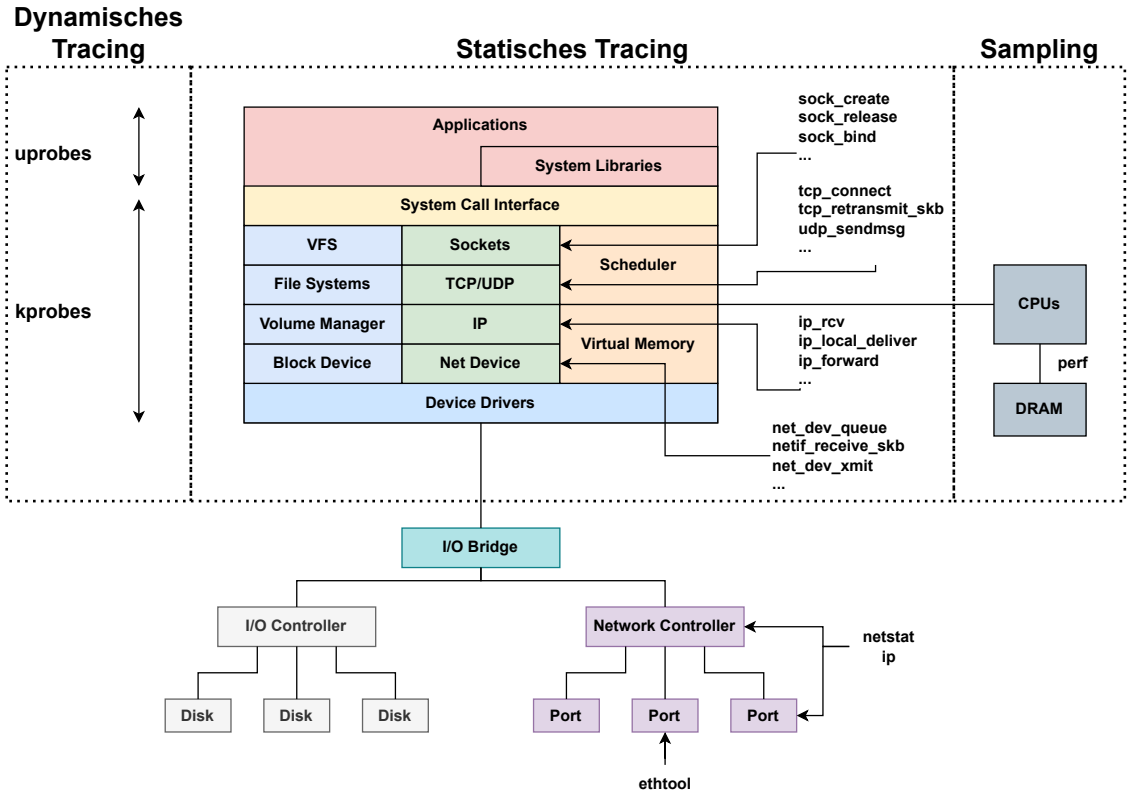

Figure 4.6 Presentation of a software stack and selection of event sources

In Figure 4.6, only three monitoring techniques are distinguished: dynamic tracing, static tracing, and sampling. In the following the available monitoring techniques are discussed, their differences highlighted and it will be explained what a probe is and how probes can be classified.

Gregg describes in [28, Chap. 1.2] that monitoring techniques serve the purpose of observability. He defines the term *observability* as the ability to understand a system by means of monitoring tools.

The term *observability* was originally introduced in 1960 by Kalman in [53]. He defines observability as the property that the internal state of a system at any given time can be completely determined from its outputs over a finite period of time.

$$x_{k+1} = Ax_k + Bu_k, \quad y_k = Cx_k + Du_k, \tag{4.1}$$

For a given state-space system model like presented in equation 4.1 the variable x_k is the state vector of the system at time step k. In equation 4.1, u_k is the input vector at time step k, y_k is the output vector at time step k, A is describing the evolution of the internal state over time, while B the input matrix describes how inputs affect the states, C is the output matrix mapping the state x_k to the measured outputs y_k, and D is the feedthrough matrix describing any direct influence of the inputs on the outputs.

According to Kalman [53], a system is observable if the *observability matrix* shown in equation 4.2 has full rank, in other words $\text{rank}(\mathcal{O}) = n$, where n is the dimension of the state vector x_k. The variables A and C in equation 4.2 have the same meaning, like in equation 4.1. The product CA^i describes how the output at time step $k+i$ depends on the initial state x_k. To i applies $i \in [0, n-1]$. The observability matrix in equation 4.2 has one column for each state variable,

in other words n columns if $x_k \in \mathbb{R}^n$.

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \tag{4.2}$$

Full column rank ensures that each state variable can be uniquely determined from the output measurements. The outputs must be multiplied by A^i in each block CA^i because the effect of the initial state on future outputs evolves over time according to the system dynamics. This condition ensures that the current state x_k can be fully reconstructed from the output measurements y_k.

Nevertheless, Gregg's definition of observability does not deviate substantially from Kalman's original formulation. A system can only be fully understood if its internal state can be determined at all times from its outputs. Also Gregg's definition is more practical orientated, where the original definition is more orientatedto the theory. In order for a system to produce outputs from which its internal state can be inferred, certain analysis techniques are required. Gregg categorizes these techniques in [28, Chap. 1.2] into *tracing*, *snooping*, *sampling*, and *profiling*.

Tracing, as described by Gregg in [28, Chap. 1.2], refers to recording the system state based on event sources. The origins of tracing can be traced back to 1951, when Gill in [54] described a method to rapidly diagnose errors in the code of digital computers. This topic is closely related to debugging, yet Gill already employed several techniques in this paper that are also relevant to tracing. He also emphasized that his error detection method should have as little impact on the system as possible. Furthermore, he employed monitoring and reacting to event sources, as can be seen in [54, p. 540, 547]. He's not using the term "tracing" in his paper yet, but he describes a method that nowerdays is considered between debugging and tracing. Like Gregg describes in [28, Chap. 1.2] nowerdays there are various tools in the Linux context that enable tracing, such as `strace` [55] or `top`. According to Gregg, `top` internally also works with event sources and subsequently outputs a summary of the information extracted from these event sources to the console. From this, it follows that tracers must, on the one hand, be capable of detecting event sources, and on the other hand, also be able to read their metadata in order to fully understand their context. Furthermore, since event sources can occur very frequently, the overhead of a tracer must remain low. Otherwise, the system would be significantly impaired.

As Gregg explains in [28, Chap. 1.2], snooping is very similar to tracing. It also describes the observation of event sources. Snooping was introduced in 1983 by Ravishankar and Goodman in [56]. They presented a bus snooping technique that monitors a memory block stored across multiple caches and informs other caches when this memory block has been modified. This approach avoids cache incoherency. Therefore, the difference between snooping and tracing lies less in functionality and more in their objectives. Snooping is generally a passive observation with occasional reactions, whereas tracing often aims to thoroughly analyze and record a specific area of the system.

The two additional analysis techniques presented by Gregg in [28, Chap. 1.2] are also closely related to each other. Sampling and profiling are both techniques that use a subset of data to represent the state of the system as accurately as possible. This approach is most commonly applied for monitoring the CPU.

Sampling also records event sources, but not all of them. Instead, it captures data only at specific time intervals. For example, sampling may take a snapshot of the current system state every three milliseconds for a duration of 0.5 milliseconds, including information such as current events or CPU utilization. Sampling is therefore well suited for contexts in which many event sources occur. Because sampling does not capture every single event source, its overhead is generally lower than that of tracing.

The recorded samples are then aggregated into a system profile. This process is referred to as profiling. By combining sampling and profiling, it is possible to represent contexts with many event sources as accurately as possible through regular snapshots.

One example of a sampling tool is `gprof`. As shown in [57], this tool can be used to perform measurements at regular time intervals in order to determine how much time the program spends in various functions.

In this work, tracing is primarily used, and therefore sampling and profiling will not be discussed further at this point. After having described the commonly used analysis techniques according to Gregg, all of which operate partially or exclusively with event sources, the following section explains what an event sources is and which types of event sources exist in the Linux context. As previously mentioned, and as shown in Figure 4.5, can be classified into static and dynamic event sources.

In Figure 4.6, the diagram is also divided into three sections, again distinguishing static and dynamic tracing. Sampling, as described is normally used when many event sources are triggered, so normally in the context of the CPU, like can be seen in Figure 4.6. Furthermore, Figure 4.6 illustrates how a system stack is organized and which areas can be instrumented with event sources. The kernel space is separated from the user space by the system call interface.

In the kernel space, event sources can be attached in the three categories memory, shown in blue, networking, shown in green, and the CPU, shown in orange. In addition, Figure 4.6 depicts the I/O controllers and the network controller, which are connected to the device drivers through the I/O bridge.

In this work, only event sources from the networking subsystem and the network controller are required. Therefore, Figure 4.6 highlights only selected events and tools from these areas. For the network controller, no native traceable events exist. However, the tools `netstat`, `ip`, and `ethtool` can be used to configure and monitor it, as described in [58, 59, 60].

Having explained where event sources can be attached, the following section discusses in detail how they operate and in which aspects they differ. The description begins with dynamic probes, as these were more readily adopted into the Linux kernel.

As Gregg describes in [28, Chap. 1.2], dynamic probes consist of Kernel Probes (kprobes) and Userspace Probes (uprobes). As shown in Figure 4.6, kprobe affects the kernel space, while uprobe affects the user space. According to Mavinakayanahalli [61, p. 101], kprobes were developed by the IBM Linux Technology Center and were integrated into the mainline kernel in version 2.6.9 in 2004. However, the mailing list [62] indicates that kprobes were patched into kernel 2.5.48 already in 2002, although they were not yet part of the mainline at that time.

4.3.2 Kprobes

A kprobe allows instrumentation of any kernel instruction or function. This is achieved, as described in [61, p. 101-104], by replacing an instruction with a software breakpoint and redirecting control to a handler. As described in [61, p. 102], a `kprobe struct` exists. This struct can be used to manage and configure probes. BPF tools, such as bpftrace, also utilize this struct in the background to register kprobes. The relevant fields include `addr`, `pre_handler`, `post_handler`, and `fault_handler`. The `addr` field specifies the location where the kprobe should be registered. The handler fields indicate the actions to be executed before, after, or instead of the probed instruction.

In addition to kprobe, there exists the Kernel Return Probe (kretprobe). As described in [61], it is used to monitor return times or return values of functions. A dedicated struct, `kretprobe`, exists for kretprobes, containing, besides the entry address, an additional `maxactive` field. Since kretprobes, as described later, are not attached to a fixed instruction address, multiple instances of the tracked function may exist simultaneously. This can occur, for example, if the function is recursive. The `maxactive` field specifies the maximum number of concurrent kretprobes allowed for a single function.

The following describes the operation of kprobes and kretprobes:

- A **Registration** is needed before use ,as stated in the official documentation [63], for both kprobe and kretprobe. This is done using `register_kprobe()` or `register_kretprobe()`. A kretprobe cannot exist without a corresponding kprobe. Therefore, registering a kretprobe always involves a kprobe. During kprobe registration, the first instruction of the target function is replaced with a breakpoint instruction which is `int3` on the x86 architecture. The original instruction is also copied to allow later execution.

- A **Trap** occurs when the CPU reaches the address specified during registration, triggering the kprobe dispatcher. The normal function flow is interrupted, and control is transferred to the kprobe handler. As described in [63], all registers are saved and control is passed to the handler via the `notifier_call_chain`.

- During the **kprobe-Handler** the `pre_handler` is executed first, followed by emulation of the original instruction, and then the `post_handler` is executed. In the `pre_handler`, the address of the kprobe struct is passed to the handler. Another function of the `pre_handler` is to initialize the kretprobe. As can be seen in [64], this is performed by `pre_handler_kretprobe(struct kprobe *p, struct pt_regs *regs)`, where `*p` is the kprobe struct and `*regs` contains registers saved during the trap. This function replaces the return address of the probed function on the stack with the trampoline address and stores the original return in the `kretprobe_instance` struct. This has the affect that the trampoline is triggered when the original function returns. Aside from this kretprobe-specific pre-handler, regular kprobe can execute user-defined code before the actual function. After the `pre_handler`, the copied instruction is executed in a single-step manner, as desrcibed in [63] Copying the instruction is necessary to avoid removing the breakpoint, which would allow other CPUs to skip the kprobe. Execution then proceeds to either the trampoline or the post-handler. In the post-handler, the return value can be further processed, or additional user code executed. For a kretprobe, the post-handler is not executed, as it is replaced by the next kprobe triggered via the trampoline. If the function terminates in a fault, such as a page fault, the previously mentioned fault handler is invoked. This handler prevents the function from crashing immediately and implements a user-defined routine for handling the error condition.

- The **Trampoline** typically consists, as described in [63], of a NOP instruction. As described by Stallings in [65, p. 545], the NOP instruction means no operation. So this operation only acts as placeholder and makes space in the instruction stream. The trampoline handler is executed and transfers control to the return handler. After execution of the return handler, the instruction pointer is restored to the original return address stored in `kretprobe_instance`. Essentially, the trampoline acts as a NOP instruction with an additional kprobe.

- The **Return-handler** corresponds to the second kprobe in the kretprobe and is used to execute user-specific code at the end of the probed function.

As described in [63], the `fault_handler` is only invoked if the probed function ends in a fault, but not if a handler itself triggers a fault. For this reason, a safety check is necessary for kprobes. This check ensures that the breakpoint instruction, which replaces the original function, lies entirely within a single function. Since the breakpoint spans multiple bytes, this is not guaranteed by default. Furthermore, it is verified that the function does not contain jumps back to the original code, which has been replaced by the kprobe breakpoint instruction. Finally, it is ensured that each instruction of the original function can be executed out-of-line.

As stated in [63], almost everything in the kernel can be probed using kprobes, except the probes themselves. There are some exceptions, which are recorded in a blacklist. This list includes all functions marked with the macro `NOKPROBE_SYMBOL()`. The blacklist can also be dynamically viewed in DebugFS under `/sys/kernel/debug/kprobes/blacklist`. It contains functions used in the kprobe workflow. For example, `do_int3` is blacklisted because probing it would cause recursion, because the kprobe would continuously probe itself. Another function on the blacklist is `do_debug`. As shown in [66] and confirmed by Intel in [67, p. 1468], this function handles the debug-exception vector #DB. It is used, among other things, for single stepping. Since kprobes use single stepping to execute the probed function, instrumenting this function would affect the handler used for single stepping. Because `do_debug` executes when a #DB occurs and the TS flag is set, which is always the case for single stepping, probing it would lead to a deadlock.

The performance of kprobes can be optimized, as described by Gregg in [28, Chap. 2.7], by replacing the break instruction with a jump instruction. This has been possible since kernel version 5.x and is applied automatically if feasible. Since a kprobe cannot instrument itself, measuring the overhead is challenging. Moreover, the required execution time depends heavily on the hardware. According to the official documentation [63], kprobes on an Intel Xeon E5410 CPU take between 0.5 and 1 μs. In contrast, kretprobes take between 0.75 and 1.75 μs. Optimized kprobes achieve significantly better results on the same hardware. For kprobes, execution times range between 50 and 60 ns, while kretprobes require between 300 and 400 ns. Babrou presents measurements of kprobes on an Apple M1 processor in [68], yielding results between 110 and 350 ns. These results suggest that the kprobes were not optimized, because the M1 CPU for Apple is about 10 years younger and contains more cores and a higher frequency, as described later.

In this work, a Raspberry Pi 5 is used. The performance of its quad-core ARM Cortex-A76 processor is closer to that of the Intel Xeon E5410 than to the Apple M1. This is due to both the number of cores and the clock frequency. The M1 chip has 8 cores and a clock frequency of 3.2 GHz, as described by Hübner in [69, p. 3]. In contrast, the Intel Xeon E5410 has 4 cores and a clock frequency of 2.3 GHz, as can be seen in [70]. As described in [71] the BCM2712 chip in the Raspberry Pi 5 also has 4 cores and a clock frequency of 2.4 GHz. Even though the Intel and Broadcom chips have similar specifications, this does not necessarily imply equal kprobe performance. The Broadcom chip is approximately 15 years newer, while the Intel chip is designed as a desktop CPU. This is reflected in the Thermal Design Power (TDP). The Intel CPU has a TDP of 80 W, and is in this way capable to use about seven times more ressources than the Broadcom chip with a TDP of 12 W. Additionally, the Intel CPU is based on the x86_64 architecture, whereas the Broadcom chip uses an ARM architecture. Nevertheless, the Intel CPU's performance metrics are much closer to the Raspberry Pi 5 CPU than to the Apple M1. It can therefore be assumed that the overhead of an optimized kprobe on the Raspberry Pi 5 is approximately 50–100 ns, whereas a kretprobe requires around 250–500 ns.

As previously described, the probes consist not only of kprobes but also of uprobes. In the following, the functioning of uprobes and how they can be distinguished from kprobes, is described.

4.3.3 Uprobes

As Corbet notes in [72], uprobes were added to the mainline kernel in version 3.5 in 2007. Thus, they were introduced approximately three years after kprobes.

As described by Gregg in [28, Chap. 2.8], uprobes are very similar to kprobes in both functionality and implementation. In contrast to kprobes, uprobes are used to dynamically trace userspace programs. Similar to kprobes, there exist both uprobes and the derived Userspace Return Probes (uretprobes), which probe the return of a function.

Unlike kprobes, uprobes are attached to a function in an executable file in userspace. As described in [73], this can be achieved by registering uprobes in the file

`/sys/kernel/tracing/uprobe_events`. For instance, using the command `echo 'p /usr/test:0x5329c0' > /sys/kernel/tracing/uprobe_events` attaches a uprobe to the executable `test` at offset `0x5329c0`. In other words, the function to be probed in `test` starts at byte `0x5329c0`. This implies that the function and offset must always be provided when registering a uprobe.

This behavior can also be seen in [74], where the function in listing 2 is defined.

```
struct uprobe *uprobe_register(
                                struct inode *inode,
                                loff_t offset,
                                loff_t ref_ctr_offset,
                                struct uprobe_consumer *uc);
```

Listing 2 Function blueprint to register a Uprobe

In listing 2, `inode` specifies the file to be probed, and `offset` indicates the offset of the function in the file where the breakpoint should be set. The argument `uc` provides information on how the probe should be handled, including callback functions for the handlers.

The beginning of the function, i.e., the location in the file pointed to by the offset, is replaced by a breakpoint instruction, similar to kprobes. However, the breakpoint instruction is not set immediately upon registering the uprobe, as noted in [75], but only when the file is loaded by a process. This ensures that the file is modified only if necessary.

As described by Zhen et al. in [76, p. 2, 3], a context switch from userspace to kernel space occurs when the CPU reaches the breakpoint instruction. Subsequently, as with kprobes, the handlers are executed in kernel space in a virtual machine. Once the uprobe has been fully executed in kernel space, another context switch from kernel to userspace occurs, and the program continues normally.

As described in [75], the instructions are loaded into a separate area. This area is implemented using a page where the instructions are single-stepped. Therefore, the instructions are executed per process. Dronamraju refers to this area, where instructions are executed per process using single-stepping, as the XOL area. In this context XOL means execute out of line. The main difference in the execution flow of a uprobe is that it requires two context switches and is executed per process. As described by Zhen et al. in [76, p. 2], these context switches can increase the runtime of a uprobe by a factor of 10.

Furthermore, at the time of this work, no optimizations exist for uprobes that are part of the kernel mainline. Optimization of uprobes by using a jump instruction, as in the case of kprobes, is not possible because, due to per process execution in the XOL area, the jump instruction would otherwise overwrite the original instructions of other processes that are not currently executing the uprobe. This is because, as described in [77, p. 3-333], the jump instruction loads the handler, i.e., the XOL area, into the current page. As a result, all processes executing the code containing the probed function would execute the uprobe handler using single-stepping. This behavior contradicts the design principle of uprobes, which is that they are executed per process. Therefore, the breakpoint instruction `int3` is used, which triggers an exception and causes a context switch into kernel space, where execution is redirected to the XOL area. Since there exists one XOL area per process, the per process design can be preserved.

Because uprobes cannot be optimized and additionally involve context switches, they are slower than kprobes. There is less documentation and fewer studies available on uprobes compared to kprobes, which is why there are only limited reliable data regarding the overhead of uprobes. Nevertheless, Zhen et al. measured an overhead of 3.2 μs in [76, p. 7]. Keniston et al. reported similar values in [78, p. 218], with an overhead of 3.4 μs. For comparison, Keniston et al. also presented results for optimized kprobes, where the overhead was reduced to 0.25 μs. This is consistent with the values given in the official documentation [63]. Thus, it can be assumed that

the overhead of uprobes is approximately 10 to 15 times higher than that of kprobes.

The performance of uretprobes is unexpectedly good in the results described by Zhen et al., with values close to 4 μs. Given an overhead of 3.2 μs, this corresponds to only a 25% increase. In contrast, the results for non-optimized kprobes in [63] showed an additional overhead of 50% to 75%, while the optimized variant reached as high as 500% to 600%. This can be explained by the fact that uretprobes do not introduce an additional context switch, which, as also described in [76], accounts for a major portion of the total execution time.

There are already initial optimization approaches that specifically target uprobes. One such approach was presented by Zheng et al. in [76]. They optimized the behavior of uprobes by reducing context switches. This is achieved by handling uprobes entirely in userspace. The approach is limited to the use of uprobes in the context of BPF. In the proposed program *bpftime*, the virtual machine for BPF is implemented in userspace instead of in the kernel. Thus, the entire execution of a BPF program, as described before, takes place in userspace. It would go beyond the scope here to explain the execution flow in detail, but Zheng et al. were able to fully implement uprobes in userspace. The per process principle was preserved in this implementation. As a result, all context switches were eliminated, which is also reflected in the performance results.

The performance of userspace uprobes and uretprobes is approximately 10 times better than that of the original probes. In [76, p. 7] values of about 314 ns for uprobes and 381 ns for uretprobes were achieved. These results are already very close to those of optimized kprobes. In particular, the values of optimized uretprobes show no significant difference compared to bpftime.

However, at the time of this work, bpftime is still a work in progress and suffers from several bugs and an unstable API, as described in [79]. Nevertheless, bpftime is mentioned here to illustrate that the performance of uprobes could soon approach that of kprobes.

After having explained what a probe is, and how it is implemented in Linux, both in the kernel and in userspace, the following section explains what an event is. Furthermore, it will be discussed how events are implemented and what differences exist compared to probes.

4.3.4 Tracepoints

An event, as Gregg describes in [28, Chap. 2.9, Chap. 2.10], serves the purpose of static instrumentation in both user space and kernel space. In the kernel space, they are implemented using tracepoints, while in user space they are realized by means of User-level statically defined tracing (USDT). The following section will first take a closer look at the implementation of tracepoints.

Tracepoints were developed in 2008 by Desnoyers. They were introduced as the successor to kernel markers, which had also been merged into mainline in 2008 with the release of version 2.6.24, as can be seen in [80]. Kernel markers are also static event sources and were developed in the context of the tracing tool LTTng, as described by Desnoyers in [81]. As Desnoyers explains in [82, p. 170], the original motivation for kernel markers was to provide an alternative to kprobes, since their overhead was too high for the development of LTTng. Tracepoints were implemented shortly thereafter as an extension of kernel markers. As Desnoyers notes in [82, p. 171], the essential difference is a stable[1] API. This ensures that both the user and the callee side have a well defined agreement about the arguments of a given function. This is implemented using the library `include/trace/events`, in which the corresponding tracepoint function can be inserted into the source code at the desired location. For this tracepoint function, both the parameters to be passed and the return values are well defined. The structure of the tracepoints can be found under `/sys/kernel/debug/tracing/events/subsystem/event/format`.

1 Tracepoints are static and therefore normally remain unchanged, but as Gregg describes in [28, Chap. 2.9], the API of tracepoints may change, although this happens only rarely.

As can be seen in [83], the first commits for tracepoints by Desnoyers were made in 2008. However, as Gregg notes in [28, Chap. 2.9], they were not released until 2009 with kernel version 2.6.32.

Gregg describes that in the kernel, tracepoints have been placed at more than 100 points of interest for monitoring. Tracepoints are continuously added by developers, which causes their number to steadily increase. On the operating system used in this work, nearly 2500 tracepoints are already available.[2]

A tracepoint follows the format `subsystem:eventname`. For example, the tracepoint `net/net_dev_xmit` consists of the subsystem `net` and the event `net_dev_xmit`. As shown in [84, Line 3831], this event is triggered when a network packet is handed over to the driver for transmission.

As described by Desnoyers in [85], a tracepoint can be in either the *active* or the *inactive* state. Gregg explains in [28, Chap. 2.9.2] that, when compiling the kernel, a 5-byte NOP instruction is inserted at the location where the tracepoint will reside. This instruction performs no operation and therefore introduces only a minimal runtime overhead. It represents the tracepoint in its inactive state.

When the tracepoint is activated, the kernel replaces the NOP instruction with a JMP instruction of also 5-bytes that points to a trampoline. In addition, a callback routine is registered for the tracepoint. The trampoline in this context is a small routine that executes all callback functions registered for this tracepoint and forwards the arguments collected by the tracepoint. Multiple callback functions can be registered per tracepoint, typically by tracers such as ftrace, perf, or LTTng. As described by Desnoyers in [86, p. 218], the results of these callback functions are usually delivered to the tracers in user space via a ring buffer.

```
1  TRACE_EVENT(event_name,
2
3      TP_PROTO(type1 arg1,
4               type2 arg2,
5               ... ),
6
7      TP_ARGS(arg1, arg2, ...),
8
9      TP_STRUCT__entry(
10         __field(type1 field1)
11         __field(type2 field2)
12         __array(char, arr, LEN)
13         __string(str_field, some_pointer_to_string)
14     ),
15
16     TP_fast_assign(
17         __entry->field1 = arg1;
18         __entry->field2 = arg2;
19         __assign_str(str_field, some_pointer_to_string);
20     ),
21
22     TP_printk("field1=%d field2=%u str=%s",
23         __entry->field1,
24         __entry->field2,
25         __get_str(str_field))
26 );
```

Listing 3 Structure of Tracepoint

2 This number can be determined with the command `find /sys/kernel/debug/tracing/events -type d | wc -l`, since all tracepoints are documented in this directory.

In the following, the structure of a tracepoint function is described.

A tracepoint is structured as shown in Listing 3. As illustrated in line 1 and explained by Rostedt in [87], the definition of every tracepoint always begins with the macro `TRACE_EVENT`. This macro consists of the following six components:

1. First, a tracepoint always requires a **name**. This name corresponds to the name of the event, as described earlier. In the previous example, this would be `net_dev_xmit`. The function invoked to place the tracepoint also contains this name. It follows the naming convention `trace_event-name`.

2. Next, the **prototype** of the tracepoint function is defined. As shown in line 3 of Listing 3 and explained by Desnoyers in [85], this prototype contains all arguments passed when the tracepoint is invoked.

3. The **arguments** used in the prototype must also be explicitly declared as parameters. While this appears contradictory and redundant, it is necessary, as described by Rostedt in [87], because the tracepoint function executes callback functions to which these arguments must be forwarded. Therefore, the arguments need to be provided again as separate parameters using `TP_ARGS`, as illustrated in line 7 of Listing 3.

4. Furthermore, the **structure** of the data passed to the ring buffer must be specified. This structure can include additional macros such as `__field(type1, field1)` or `__array(char, arr, LEN)`, as shown in lines 9–14 of Listing 3. Here, `type1` could, for example, be an `int`, and `field1` is the name of the variable. For arrays, the same applies, with the additional parameter `LEN` which is specifying the number of elements.

5. In addition, the **assignment** must define how the data is stored in the ring buffer. As explained by Rostedt in [87], this step specifies how the previously defined structure is populated with the arguments introduced in step 2. This is implemented using the parameter `TP_fast_assign`, as shown in line 16 of Listing 3.

6. Finally, the **output** is defined using `TP_printk`. As Rostedt explains in [87], this represents a format specification for the output of the fields contained in `TP_STRUCT__entry`.

Having described how tracepoints function and the fact that they were developed to deliver better performance than kprobes, the following section will focus on the performance of tracepoints.

Also for tracepoints, there are only very few reliable data regarding their overhead time. A distinction must be made between inactive and active tracepoints.

As Edge describes in [88], inactive tracepoints incur an overhead of approximately 2-4 cycles. At a clock frequency of 2.4 GHz, as provided by the Raspberry Pi 5, this would correspond to a permanent overhead of about 1–1.5 ns. In the active state, both Zheng et al. in [76, p. 7] and Gebai in [89, p. 26:19] measure an overhead of 100–150 ns. However, this exceeds the overhead of the optimized kprobe, which incurs only around 60 ns.

In contrast Gregg, in [28, Chap. 2.9.5], reports different figures. He compares the overhead of kprobes with tracepoints and raw tracepoints. Raw tracepoints were introduced in Linux 4.17 in 2018 and do not provide a stable API, but only a stable tracepoint name. This reduces the overhead slightly. According to his results, raw tracepoints exhibited a maximum overhead of 9%, regular tracepoints up to 14%, and kprobes up to 30%. However, he does not provide any information on whether the kprobes used in the measurements were optimized or not. These results contradict the previously mentioned findings. For optimized kprobes they appear too poor, while for unoptimized ones they appear too good.

Desnoyers reports in [82, p. 94] that kprobes require 1.49 μs. This also explains why tracepoints were developed as an evolution of kprobes. Optimized kprobes were only added to the kernel at

the end of 2009, as indicated in [90]. Therefore, at the time of the development of tracepoints, they were not yet available.

There are no reliable data indicating whether optimized kprobes or tracepoints exhibit lower overhead. However, in both cases, the overhead is very small.

Besides the tracepoints for the kernel space, there also exist USDTs for the user space. In the following section the userspace equivalent of tracepoints will be explained,

4.3.5 User-level statically defined tracing

As Gregg describes in [28, Chap. 2.10], they work in a very similar way to kernel tracepoints. USDTs were developed to enable static tracing in user space. The functionality was first implemented in 2004 as part of DTrace [91, 92], but was only added to Linux later. From 2016 onwards, an integration with BPF was available through BCC.

Technically, at the designated location in the program, a USDT is placed, which initially exists as a NOP. When the probe is enabled by a tracer, the kernel tracing subsystem replaces the instruction with a software breakpoint. Through this breakpoint, execution switches to the kernel, where the corresponding eBPF program or perf event is triggered.

The processing of data does not take place via explicit callback functions in user space, but rather through the perf_event infrastructure, as described by Gregg in [28, Chap. 2.13]. A file descriptor is opened via `perf_event_open()`, through which data is written into the perf ring buffer, as shown in [93].

In contrast to uprobes, no currently running instruction needs to be replaced or emulated when patching, which makes USDTs more efficient. However, even with USDTs, a transition into the kernel is always necessary when the tracepoint is triggered, and thus a context switch occurs.

As described in [91], USDTs can be defined in user code via the library `#include <sys/sdt.h>`. A tracer must then be attached to activate them.

There are only very few data sources available regarding the overhead of USDTs. According to [89, p. 26:28], the overhead varies between 80 ns and 450 ns depending on the tracer. This is the only source that provides overhead measurements for USDT. The values are considered reliable, as the same source already provides reference values for kernel tracepoints. However, no specific data for eBPF was given. Nevertheless, it can be seen that the overhead is significantly lower than that of a uprobe. The order of magnitude lies between a factor of 10 and 40.

The overhead of a non-activated USDT remains the same as that of a kernel tracepoint and is therefore between 1 ns and 1.5 ns.

After various event sources[3], both static and dynamic, have been explained, the question arises, which is more suitable for monitoring? This will be presented in the following section.

4.3.6 Comparison of various event sources

In Table 4.1, the most important characteristics of the described event sources are summarized. The four event sources are compared in terms of type, overhead in active and inactive states, as well as maintainability, stability and flexibility.

As shown in Table 4.1, the static variants offer less flexibility. This means that fewer probe points are available, both in the kernel and in user space, where they can be attached. In addition, even when inactive, a small amount of overhead remains. Furthermore, maintenance effort is required to ensure that they remain available in all future versions of the program or the kernel.

3 Not all event sources were described, but only a selection of the most common ones and those relevant for this work. Gregg additionally describes dynamic USDTs and performance monitoring counters in [28, Chap. 2.11, Chap. 2.12].

Despite these disadvantages, the static event sources also provide several important advantages. First, a stable API exists that defines exactly which information is made available by a tracepoint. Moreover, tracepoints are preserved in future kernel versions. A performance improvement over dynamic event sources[4] can also be observed.

On the kernel side, tracepoints are similar as fast as optimized kprobes. Since kprobes, however, cannot always be optimized, the overall performance of tracepoints is generally better. On the user-level side, the performance of USDTs is significantly better than that of uprobes. The performance of USDTs is only slightly worse than that of tracepoints and is therefore, depending on the tracer, between 10 and 40 times faster than a uprobe.

Table 4.1 Comparison of the event sources kprobes, uprobes, tracepoints, and USDTs

Event source	**kprobe**	**uprobe**	**tracepoint**	**USDT**
Type of event source	dynamic	dynamic	static	static
Overhead (inactive)	0	0	~1.5 ns	~1.5 ns
Overhead (active)	500 – 1750 ns (50 – 300 ns)[a]	3000 – 4000 ns	100 – 200 ns	60 – 450 ns
Stable in different kernel versions	No	No	Yes	Yes
Count of available events	60000+	nearly all symbols in user space	2400+	only predefined tracepoints
Maintenance	No	No	Yes	Yes

a Values in parentheses refer to optimized kprobes.

From these data, it follows that static event sources require more effort to maintain and to implement. However, this effort is compensated by better performance and higher stability.

This observation is also supported in the literature. Gregg [28, Chap. 2.9] explains that tracepoints should always be used first whenever possible. Kprobes should only be employed as a fallback when no tracepoints exist for a specific use case. Gregg's opinion is further supported by Calavera et al. [46, Chap. 4].

For monitoring in user space, uprobes are also considered insufficient, as argued by Zheng et al. [76, p. 3]. They explain that, due to the relatively high overhead of uprobes and the additional required context switches, these are not suitable for time-critical or real-time monitoring tasks.

In summary, considering the advantages and disadvantages of the respective event sources, it is considered best practice to prefer static ones. If this is not possible due to lack of support or disproportionately high effort, then, as also mentioned by Calavera et al. [46, Chap. 4], dynamic event sources can and should be used.

Since the basics to monitoring and the functionalities of event sources, which are needed for the implementation, were described, the following chapter focuses on the implementation details of the test system itself.

4.4 Test system

In this chapter, the test system is divided into the four components test application Server and Client, Test Automation and Access Point Configuration. Since for the implementation of the client and server the test application, which is generating the data is required, the test application wil be described first. Afterwards the implementation of the client and server will be

4 For the overhead of dynamic event sources, the minimum values correspond to the minimal reported latency of a simple probe, i.e. a kprobe or uprobe, and the maximum values correspond to the maximum reported latency of the respective return probes.

explained. Subsequently, the focus shifts to test automation, and finally to the creation of the access point.

4.4.1 Test Application

First, the requirements that the test application must fulfill are discussed. Afterwards, the test application itself is explained, and it is described how it meets these requirements.

For the test application, the following three aspects are relevant:

1. **Real-time capability and execution time**: The focus of this work is to evaluate influencing factors on the real-time capability of Wi-Fi. Therefore, influencing factors that are not related to Wi-Fi should be minimized. If the execution time of the application is too long or if it requires many shared system resources such as the CPU, cache, memory, or graphics card, then the application becomes vulnerable to real-time issues. That resource contention is a common problom for real-time systems, as stated among other by Aceituno et al. in [94, p. 1]. Especially in a multi-core environment it is not possible to avoid resource contention completely, but to restrict it as far as possible. It is mandatory that the application achieves real-time capability, since real-time violations would otherwise directly affect the test results. Furthermore, it is necessary that the application in general does not consume too much time, since otherwise the previously defined deadline of 3 ms, maybe cannot be met, or this at least becomes more difficult, as less time remains available for the Wi-Fi operation.

2. **Amount of data**: It is also required that the application continuously generates data that can be transmitted. This ensures that a sufficiently large number of data transmissions take place, which in turn are the foundation for reliable test results. In other words, if the application only generates two data sets, only two transmissions occur. From such a small number of transmissions, no reliable conclusions regarding the influencing factors on Wi-Fi performance can be drawn.

3. **Visualization**: This point is rather a benefit than a strict requirement. If real-time violations within the application can be visualized, the results become more intuitive and easier to interpret. While this is not strictly necessary, it can enhance the comprehensibility of the results.

In Figure 4.7, the concept of the test application is presented. It is divided into the three sections Application Concept, Calculation, and Visualization. The discussion first addresses the Concept.

The concept consists of two main components, namely the Network Model and the Job. The Network Model specifies which jobs are executed and at what time. It is inspired by the theory of scheduling workload models, as described by Baruah et al. in [95, p. 14–20]. Although this work does not investigate the real-time capability of scheduling itself, the models can nevertheless be applied in an adapted form to the network.

In scheduling theory, jobs are also used to represent workload models. There a job represents an application that is executed by a thread. Each job is characterized by three properties:

- The **arrival time**, which specifies when the job begins. It corresponds to the point in time when the event `scheduleswitch` is triggered, where the parameter `nextpid` contains the identifier of the thread.

- The **deadline**, which is the latest point in time by which the job must be completed. The interval between arrival time and deadline is referred to as the scheduling window and represents the time available for execution.

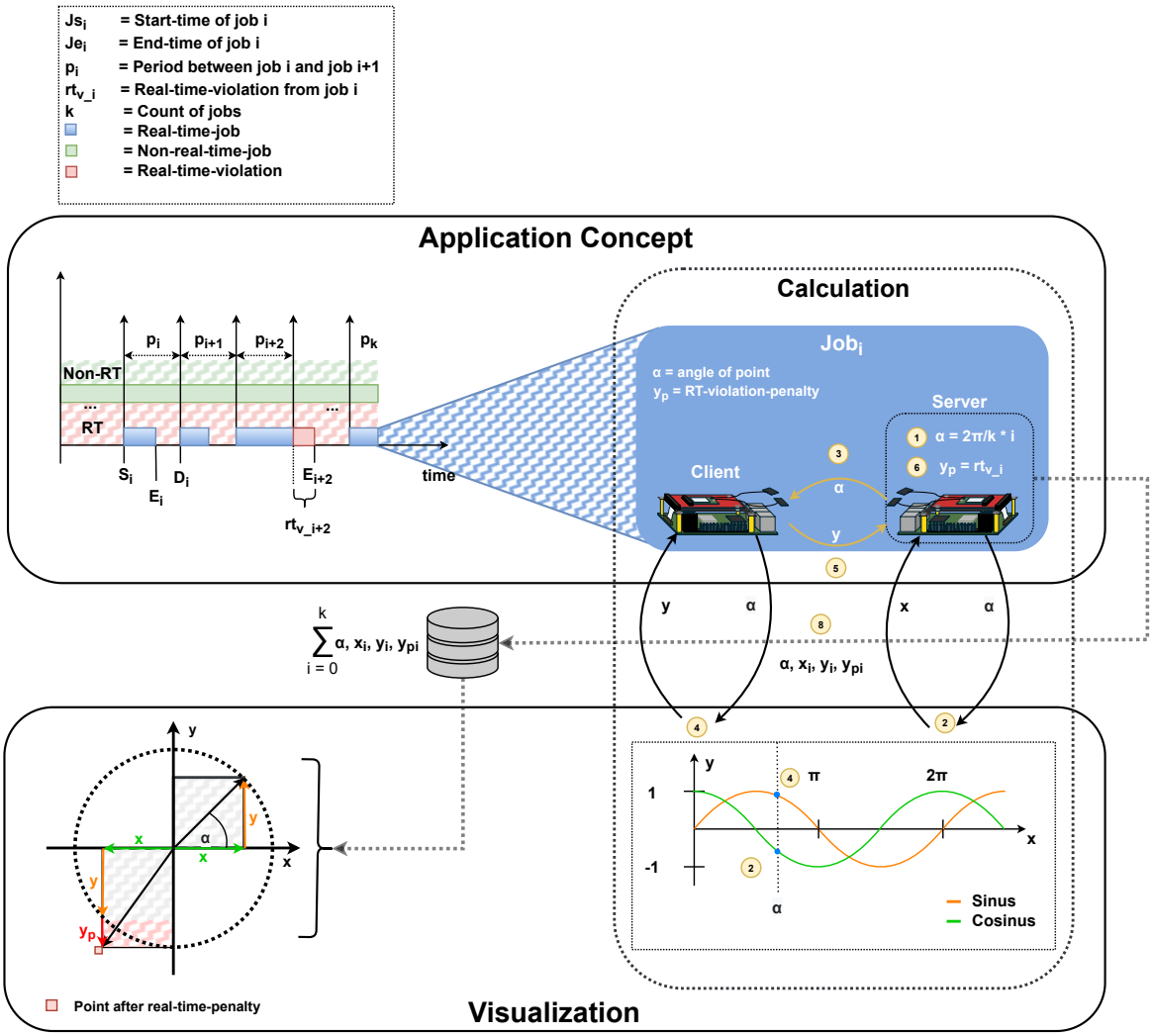

Figure 4.7 Presentation of the concept and visualization of the test application

- The **Worst-case execution time (WCET)**, which denotes the maximum execution time required by the job. It corresponds to the execution time in the worst case, i.e., when the job is executed under the most unfavorable conditions. Such conditions may arise due to external influences such as interrupts or blocked resources. In the networking context especially the time in the networkstack can be considered as a part of these conditions. The WCET must be smaller than the scheduling window to guarantee that the job finishes before its deadline, thereby ensuring that the application represented by the job is real-time capable.

These properties can also be transferred to the application in this work. Although scheduling itself is not the subject of observation, the underlying theory remains the same. By assigning a given period, the job can be repeated multiple times, allowing conclusions to be drawn about the behavior of the network.

As illustrated in Figure 4.7, the periodic model of Liu et al. has been applied, originally published in 1973 in [96]. This model has been used for more than fifty years to evaluate the real-time properties of applications. The model assumes an implicit deadline, which is defined by the length of the period.

A job J_i starts at its arrival time S_i and should complete by its deadline D_i. In the case of an implicit deadline, the deadline D_i is equivalent to the arrival time of the subsequent job, i.e.,

S_{i+1}. These time points are always separated by the period p_i. The period usually remains constant, such that $\forall i, j \in [0, k]$, $k \in \mathbb{N} \setminus \{0\} : p_i = p_j$. The period therefore also defines the length of the scheduling window.

Besides the original workload model proposed by Liu et al., there exist other models, such as the three-parameter sporadic task model, which is introduced by Baruah et al. in [95, p. 15, 16]. This model is structured similarly to the periodic model but differs in two essential aspects.

First, it includes an explicit parameter for the deadline, which allows the deadline to occur either before or after the release of the next job. In other words, the constraint $D_i = S_{i+1}$, which always holds in the model of Liu et al., does not apply here.

The second major difference is that the period length p_i is no longer fixed. Since the sporadic model is not fully periodic, a stochastic component is introduced. As a result, the model only guarantees a minimum interarrival time $p_{\min}$ between two tasks. However, it is not ensured that S_{i+1} occurs exactly or shortly after $p_{\min}$ time units following S_i. Consequently, the model exhibits both periodic and sporadic characteristics.

For evaluating the real-time capability of an application, the periodic factor is highly relevant, since otherwise the data set might be too small to enable result-based conclusions.

In the context of network communication, there also exist two models, as described by Khojasteh et al. in [97]. These are categorized into the time-triggered and the event-triggered models. They define the conditions under which network data is transmitted in an application.

In the time-triggered variant, data is sent at fixed time intervals, analogous to the periodic task model. In contrast, the event-triggered variant transmits data only when a specific event occurs, which resembles the sporadic task model. According to Khojasteh et al. [97, p. 6], the event-triggered approach requires less bandwidth and, for the same stability requirements, achieves a lower transmission frequency compared to the time-triggered approach. Thus, the event-triggered model is described as the more efficient option.

However, this view is contrasted by the study of Meister et al. in [98], where the two models are also compared. Their results demonstrate that in practice, especially under network effects, the time-triggered model can outperform the event-triggered one. Since the studies of Khojasteh et al. where only theoratical and the network load were not considered, the results of Meister et al. can be considered as meaningfull.

Since this work also considers network effects, and because the periodic model provides lower complexity, improved observability, and enhanced predictability, a periodic model is adopted in this thesis. The periodic model has a lower complexity, because it has no separate parameter for a deadline or a minimum period and therefore less parameters in general. It can also be monitored more easily, because it is more predictable. It can be determined exactly when the next job will occur and when all jobs will be completed.

In Figure 4.7, the jobs are shown in a model that is very similar to the one of Liu et al. A distinction is made between real-time jobs, which are shown in blue, and non-real-time jobs, which are shown in green. It can be observed that only the real-time jobs possess a period duration and an arrival time. The non-real-time job extends across the entire timeline. This is because there is no specific regulation for it. The requirement was to transmit as much bulk data as possible alongside the real-time data, which results in these data being sent continuously. Only the real-time jobs are subject to timing constraints. For J_{i+2}, the model also visualizes how a real-time violation occurs. J_{i+2} finishes at the time E_{i+2}, which is the interval rt_{v-i+2} after its deadline D_{i+2}. This interval is shown in red. Thus, the diagram does not represent the load on the CPU, but instead the load on the network, at least in part.

The execution of a job is depicted on the right side in the Application Concept. A job is one iteration in which a step of the application is executed. The application visualizes a unit circle. In the following, it is described how this circle is constructed. The entire coordination is performed by the server, while the client has no information other than what the server provides. The execution of the job is labeled with numbers, starting with (1) which represents S_i. The

server is managing this, by starting a timer. In the first step, also the angle α is calculated, which determines the point on the circle. It is computed using $2\pi \div k \times i$ and is shown at the bottom left of the circle. Here, $2\pi \div k$ denotes the step size and i the step number. The previously described variable k defines the number of points on the circle and thus also the number of jobs.

In step (2), the server computes the x-coordinate of the point by evaluating $cos(\alpha)$. This process is illustrated in the Calculation box. In step (3), the server sends a message to the client. This message contains the angle α and the radius of the circle. In step (4), after receiving the network packet with α, the client computes the y-coordinate of the point. This is again shown in the Calculation box and is obtained using the function $sin(\alpha)$. Then, in step (5), the client sends a packet back to the server containing the y-coordinate. In step (6), the job, as shown in the workload diagram, ends. As soon as the server receives the client's message, it stops the timer and checks whether the deadline D_i was met. If the deadline is met, the penalty value y_p is set to 0. If the deadline is missed, a penalty is applied. As described in Section 2.7, the value of a real-time application does not only depend on the result but also on the timing constraints. Therefore, y_p was introduced. Even if D_i were missed, the result would still be correct. However, in order to visualize the real-time violation, the result is deliberately distorted by y_p, since according to the definition of a real-time application it must be considered incorrect.

In step (7), the angle α, the calculated coordinates (x_i, y_i), and the penalty y_{pi}, which is 0 if the real-time constraints are satisfied, are stored in a pre-allocated array. This is shown in the middle of Figure 4.7. In step (8), after all jobs are completed, the results are visualized. This ensures that the real-time capability of the application is not restricted. The effect of y_p on the visualization can be seen in the lower left of Figure 4.7, where a red square marks the new point that no longer lies on the circle. The Visualization box shows the unit circle and how it is composed of the sine and cosine functions.

After describing the job execution, it becomes clear that in this modified model, unlike in the original scheduling workload model of Liu et al., a job is not an executed application, but rather one iteration in a sequence of instructions within an application. This sequence of instructions also includes two network communications, one from the server to the client and one back.

After describing how the application is structured and how it functions, the following section will discuss how it is technically implemented.

4.4.2 Server and Client Implementation

As described before, the entire logic is implemented on the server side, and the client only reacts to the messages of the server.

The application, as illustrated in Figure 4.7, constitutes only a small part of the overall server.

The logic of the server can be represented as a state machine, as shown in Figure 4.8. The state machine is deterministic, since, as described by Rabin et al. in [99, p. 120], a deterministic state machine requires that for a given state and a given operation the target state must be uniquely defined. This property holds for the state machine in Figure 4.8.

Furthermore, the state machine has finitely many states, which according to [99, p. 114] is the defining property of a finite state machine. Consequently, the state machine in Figure 4.8 is a Deterministic Finit State Machine (DFSM).

It can be observed that the server consists of nine states.

The application, as described in Section 4.4.1, occupies only two of these states. Steps (1) to (7) of Figure 4.7 are executed in the state *Calculation*, and Step (8) is executed in the state *Save*.

The states in Figure 4.8 can be classified into four categories. First, there are the start and end nodes, as is common for finite state machines. Second, the regular states in Figure 4.8 are distinguished into states without real-time requirements, shown in green, and states with real-time requirements, shown in red. It can be seen that the automaton starts in the state *Setup* and can terminate in the two states *Error* and *End*. Furthermore, only the state *Calculation* has real-time requirements.

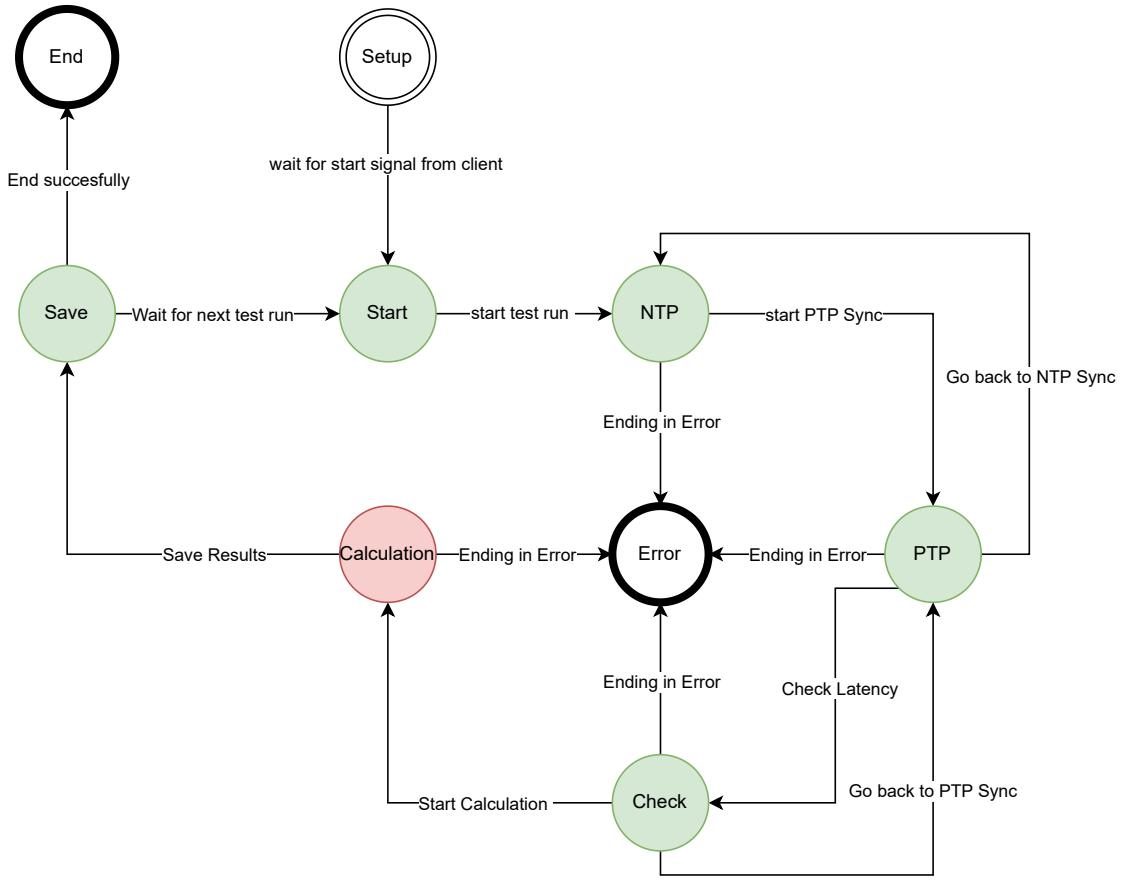

Figure 4.8 Representation of the server state machine. Green states have no real-time requirements. Red states are subject to real-time constraints.

The following describes the function of each state and how it is implemented.

The state *Setup* is the initial state. It sets the priority to real-time priority, loads and initializes the BPF program, and starts the server. As shown in Listing 10, line 3, the real-time priority is set to 99, which, as described in Section 2.7.2, corresponds to a priority of 139. For monitoring, BPF is used, which is integrated into Rust through `libbpf_rs`. As described in [100], the Rust code resides in the `/src` directory, while the BPF code, written in C, resides in `/src/bpf`. The BPF file must have the extension `.bpf.c`. In order to integrate the BPF program into the Rust code, a skeleton must be generated. A skeleton is an automatically created binding layer that provides a type-safe interface to the maps and programs defined in the compiled BPF object file. It abstracts away low-level details of loading, attaching, and accessing kernel objects, and thus enables the interaction from rust with the BPF subsystem in a safe and idiomatic manner.

As shown in Listing 10, line 1, this skeleton is subsequently included in the Rust code.

In order to generate the skeleton, a build script must be written, which, as described in [101], has to be placed in the file `build.rs`. This integration ensures that the creation of the skeleton becomes part of the Rust compilation process.

The build script is presented in Listing 4. In this script, the path to the kernel headers must first be determined, since the BPF program has to be compiled against them in order to make use of kernel structures, such as tracepoints. To identify the kernel headers, the script attempts to read the path from an environment variable, as shown in lines 5 and 6. If this attempt fails, the default path `/usr/src/kernel` is assumed.

Subsequently, the skeleton can be generated by means of the `SkeletonBuilder`. As indicated

in [102], the builder requires three arguments. First, it requires the BPF source file, which in this case is `monitore.bpf.c`, as illustrated in line 8 of Listing 4. Second, it requires the output file, that is, the file name in which the skeleton will be stored after generation. This is shown in line 15. Finally, the builder requires compile flags for `clang`. These include, among others, the kernel UAPI headers, as depicted in lines 9–14.

```
1  use std::env;
2  use libbpf_cargo::SkeletonBuilder;
3
4  fn main() {
5      let kernel_headers = env::var("KERNEL_HEADERS").
6          unwrap_or("/usr/src/kernel".to_string());
7      SkeletonBuilder::new()
8          .source("src/bpf/monitore.bpf.c")
9          .clang_args(&format!(
10             "-I{}/arch/arm64/include/generated/uapi
11             -I{}/include
12             -I{}/include/uapi
13             -I{}/include/generated",
14             kernel_headers, kernel_headers, kernel_headers, kernel_headers))
15         .build_and_generate("src/bpf/monitore.skel.rs")
16         .unwrap();
17 }
```

Listing 4 Rust Build Script

Using the skeleton, as illustrated in Listing 10 line 5, an `ObjectBuilder` is initially created through `MonitoreSkelBuilder::default()`. This builder can subsequently be opened using the `.open()` method.

During this step, the compiled BPF object file is "opened," resulting in the creation of an `OpenObject`. In other words, the BPF program is preloaded, thereby enabling modifications to the BPF program as described in [100]. However, such modifications are not required in this context. Consequently, the `OpenObject` can be loaded directly via the `.load()` method, as demonstrated in line 6.

At this point, the compiled BPF program is loaded into the kernel and passes through all mandatory verification processes, including the verifier. This procedure is further elaborated in Chapter 4.3.1.

Once successfully loaded, the resulting `Object` enables interaction with the BPF script residing in kernel space. Accordingly, as shown in lines 7 and 9, all programs specified in the BPF file are attached to their designated locations within the kernel. In other words, the defined programs are bound to the corresponding event sources. Additionally, the maps defined in the BPF file are retrieved to enable access to the data generated by the BPF programs.

Once the BPF file is operational in kernel space, a ring buffer is created in lines 11–14, to which a callback function is attached. Although Gregg [28, Chap. 2.1] describes communication with the user space via a perf buffer, this work employs a ring buffer.

As described in [103], the ring buffer is the successor of the perf buffer and was introduced in Linux kernel version 5.8 in 2020. Liu et al. [104, p. 6] report that it provides up to 25% lower access latency. At the time of publication of [28], the ring buffer was not yet mainline, which explains why Gregg uses the perf buffer.

In lines 16–20, a polling thread is launched, which continuously checks for the presence of new events. As detailed in [105], polling continues until a new entry is available in the ring buffer or a timeout of 3 milliseconds elapses. This timeout is chosen to ensure that, in the absence of new events, the overall process is not blocked, as the application operates on a periodic workload

model with a 3 ms interval.

In lines 22–36, a Transmission Control Protocol (TCP) server is initialized, and the `SetupContext` is configured. The `SetupContext` is a structure encompassing all data required in subsequent states. After its initialization, in line 31, the main program flow, managed by the `run_state_machine()` function, is started. The setup is identical for both server and client, with the exception that the client initializes a client object to connect to the server, whereas the server sets up a TCP server.

In the *Start* state, which is illustrated in Figure 4.8 and represents the next state after setupt, the server waits until the client has established a connection and transmitted the start message.

For the communication, different message types are employed. These are shown in Listing 5.

The attribute `#[repr(u8)]` specifies that the enumeration `MessageType` is represented internally as an unsigned 8-bit integer as described in [106]. The attribute `#[derive(Serialize, Deserialize, Debug, PartialEq)]`, like state in [107], instructs the Rust compiler to automatically generate implementations for the traits `Serialize`, `Deserialize`, `Debug`, and `PartialEq`. This allows instances of the enumeration to be easily serialized and deserialized for communication purposes. For this the crate `bytemuck` is used, which is described in [108]. This is necessary to reduce memory allocations and to ensure the correct layout of the message structures.

```
1 #[repr(u8)]
2 #[derive(Serialize, Deserialize, Debug, PartialEq)]
3 enum MessageType {
4     Start = 0,
5     NTP = 1,
6     PTP = 2,
7     Check = 3,
8     Calc = 5,
9 }
```

Listing 5 Message Types

In the *Start* state, the server waits until it receives a message of type `Start`. Subsequently, it transitions into the next derivephase of the protocol.

After the state *Start*, the three states Network Time Protocol (NTP), Precision Time Protocol (PTP), and *Check* follow. All three states serve the purpose of time synchronization. Contrary to what their names suggest, NTP and PTP are not directly applied. Instead, the names merely describe the functionality of the corresponding synchronization protocols. Accordingly, in the NTP phase a coarse synchronization between client and server is performed. In contrast, in the PTP phase a finer synchronization is carried out.

To understand why synchronization is necessary, the exact implementation of the monitoring process is first considered. The objective of the monitoring is to represent, as precisely as possible, the duration required by the program in specific parts of the data transmission process. These include the program execution itself, the network stack, and the transmission time within the medium. The schematic representation of the monitoring process is shown in Figure 4.9.

The figure illustrates both the server and the client. Furthermore, the application is depicted as a simplified part of the network stack within the kernel space as well as on the hardware level. The execution flow of the application, which was described in Section 4.4.1, is also included in the figure. Measurement points are highlighted at specific positions, and the resulting time intervals are illustrated on the left-hand side of Figure 4.9. A distinction is made between eight intervals, denoted as t_1 through t_8. In the following, it is discussed how these intervals are measured and how they are composed. It is assumed that the application starts on the server. The intervals are additionally marked with arrows in the corresponding colors in Figure 4.9.

For the measurements, tracepoints are used, since they provide the advantages described in Section 4.3.

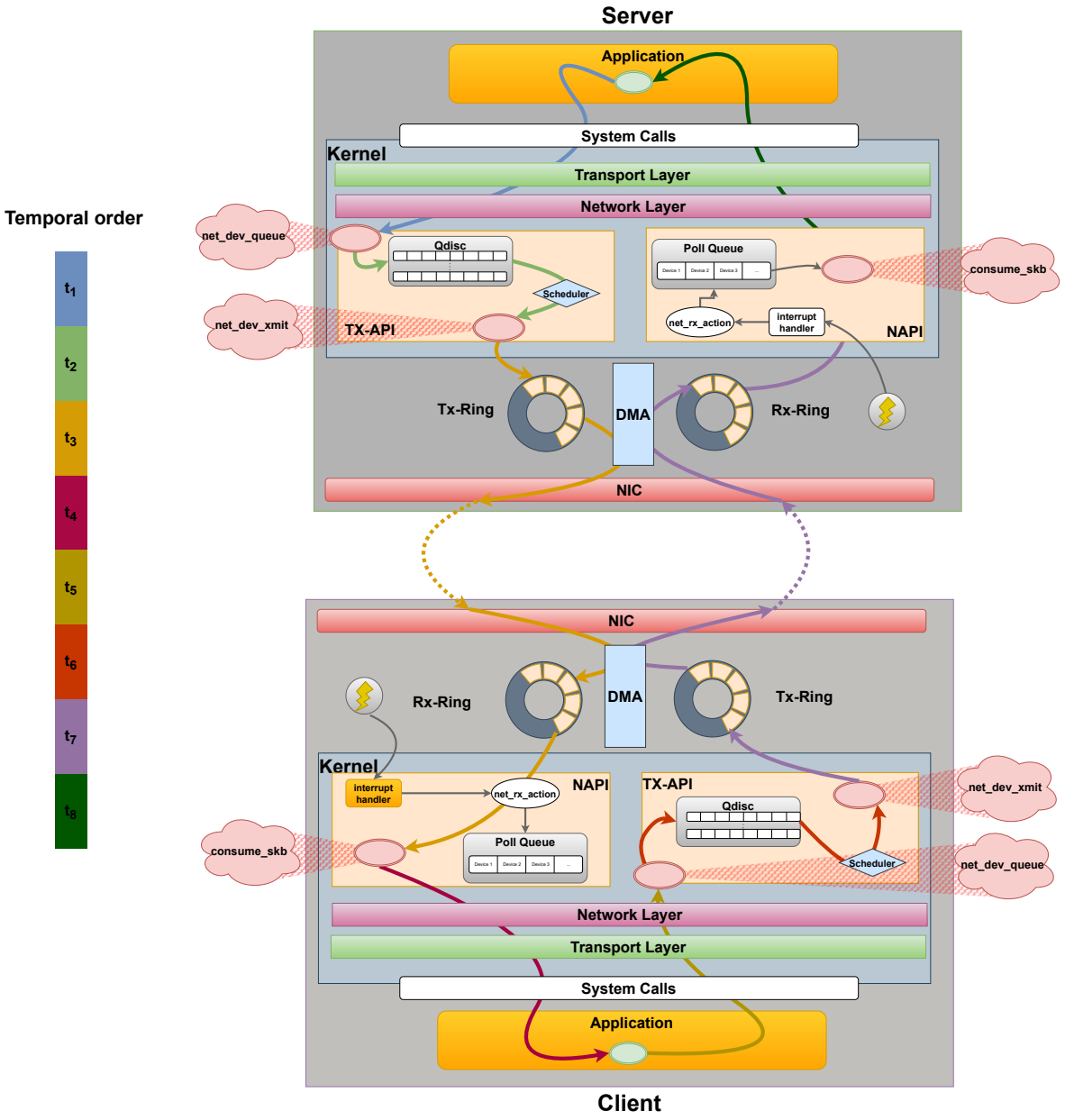

Figure 4.9 Presentation of the temporal order of the application in the network stack, adapted from [1].

- Interval t_1 starts in the application on the server, i.e., at position (1) in Figure 4.7. It ends at the tracepoint `net_dev_queue`, which marks the entry into the Queueing Discipline (Qdisc) and thus the TX-API. The TX-API is described in detail in Section 2.3. This interval therefore measures the time the application requires to compute the X-coordinate and to transfer the packet from user space to kernel space. As shown in Figure 4.9, the packet already traverses the transport and network layers and ends at the entry into the link layer. This point is of particular interest, since packets can accumulate in the Qdisc and an unfavorable scheduler may delay real-time packets. Therefore, this point is also relevant to enable the measurement of the Qdisc. Moreover, it includes a context switch which, as described in Section 4.3, introduces a non-negligible latency.
- Interval t_2 begins at the end of t_1, at the tracepoint `net_dev_queue` and ends at the

tracepoint `net_dev_xmit`, which marks the exit from the Qdisc and thus the entry into the TX ring buffer of the driver. This tracepoint represents the last measurement point on the server side before a network packet leaves the server. Hence, interval t_2 measures the time the packet spends inside the Qdisc. This is particularly relevant to detect potential congestion in the Qdisc. Furthermore, it allows the offset introduced by specific queuing strategies to be determined. As reported by Corbet in [109], the locking mechanisms of the Qdisc alone introduce a delay of 48 ns. He further states that the average delay caused by the Qdisc is in the range of 58–68 ns.

- Interval t_3 starts at the end of t_2, i.e., at the tracepoint `net_dev_xmit`, and ends at the tracepoint `consume_skb` on the client. This interval therefore includes the time required for a packet to be transferred from the TX ring buffer of the server via Direct Memory Access (DMA) to the server's Network Interface Card (NIC), to be processed there, transmitted over the medium, and received by the client. On the client side, the packet must be received by the NIC, transferred again via DMA into the RX ring buffer, and then passed into kernel space. As soon as a new packet is available in the RX ring buffer, an interrupt is triggered, as illustrated in Figure 4.9. This interrupt activates the client's poll queue via the soft IRQ `net_rc_action`. A device in this queue triggers the tracepoint `consume_skb` to read the packet from the buffer. This is the first tracepoint which is triggered when a network packet arrives at a device. Thus, interval t_3 covers the time from the server's link layer through the physical layer, across the medium, into the client's physical and link layers, more precisely into NAPI. The terms NAPI, RX ring, and soft IRQ are explained in Sections 2.3 and 2.7.2. This interval involves multiple layer transitions of the Open Systems Interconnection model (OSI-model), as well as critical points such as the RX and TX ring buffers of the drivers and the processing in both NICs. However, no further tracepoints exist in these areas to allow finer-grained monitoring.

- Interval t_4 starts at the client's `consume_skb` tracepoint and ends when the packet reaches the application in the client. Thus, it covers the time from the client's link layer, through the network and transport layers, into the application. In this phase, a context switch from kernel space to user space and a system call are required. Both the context switch and the system call may vary in duration depending on system load.

- With this interval, half of the application flow is completed, and the intervals repeat, but this time from client to server. Interval t_5 corresponds to interval t_1 and begins in the client application as soon as computation starts. This interval ends at the client's tracepoint `net_dev_queue`. Interval t_6 also takes place entirely on the client side, ranging from `net_dev_queue` to `net_dev_xmit`, thus measuring the client's Qdisc delay. Interval t_7 starts at `net_dev_xmit` on the client and ends at `consume_skb` on the server. Hence, it measures the time required to transfer the packet from the client's link layer to the server's link layer. Interval t_8 closes the cycle by measuring the time from `consume_skb` to the application on the server, thereby reflecting the time from the link layer to the application. From this point, the measurement cycle restarts with interval t_1.

Given the large number of layer transitions and the transmission time over the medium, it can generally be expected that intervals t_3 and t_7 will constitute the largest share of the total time. For the intervals t_3 and t_7, time synchronization between client and server is required, so that at time τ_1, corresponding to the tracepoint `net_dev_xmit` on the server and at time τ_2, corresponding to the tracepoint `consume_skb` on the client, a common time base is available. Otherwise, these values could not be compared with each other.

It is furthermore necessary to synchronize time measurement between kernel space and user space on both devices. This is required for the intervals t_1, t_4, t_5, and t_8, since in these intervals measurements are taken on both the client and the server, from the application to the tracepoint

net_dev_queue and from the tracepoint consume_skb back to the application. Thus, for all four intervals, a time measurement in the kernel at the tracepoint and a time measurement in user space within the application are required.

The synchronization between user space and kernel space is addressed first, since it is relevant for the subsequent synchronization between client and server. In BPF, that is, in the code of the application executed within the kernel, time can only be measured using the function bpf_ktime_get_ns(). This function returns the time in nanoseconds since system start, as described in [110].

In Rust, that is, on the user space side, time can be measured using several functions, as described in [111]. The function std::time::SystemTime::now() also returns the time since system start. However, this value does not match the time obtained from BPF. Even though both clocks are monotonic, BPF refers to the uptime of the kernel, whereas Rust refers to the uptime of the user space. Furthermore, std::time::SystemTime::now() is not reliable, as discussed in [112], since the monotonic clock may refer to different reference points in time. Rust does not rely on a single system start time, but on several closely related reference points. Although these points are very close to each other, they may still cause measurable drift when using std::time::SystemTime::now(), like stated in [112].

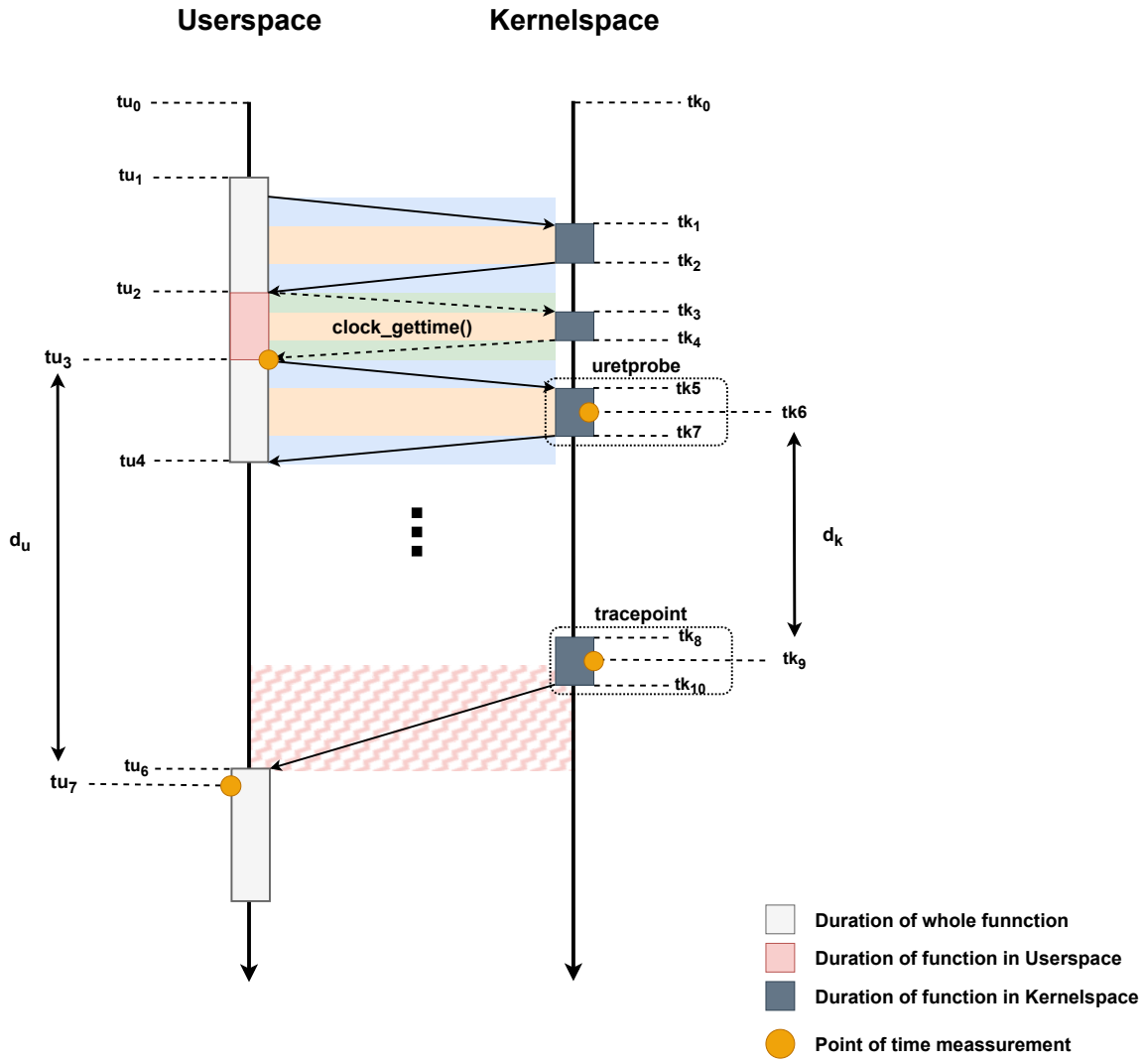

Figure 4.10 Presentation of the synchronization between User- and Kernelspace

The exact composition of the difference between the two time measurements will be described later in this chapter. The implementation of the User- and Kernelspace synchronization is shown in Figure 4.10. For synchronization, a uretprobe is used. The goal is to establish a common time base simultaneously in user space and kernel space by means of the uretprobe. This common base then allows the times measured in user and kernel space to be related to each other.

As described in Section 4.3, a USDT is more suitable than a uprobe for performance reasons. Zheng et al. also state in [76] that a uprobe is not real-time capable and that its overhead is particularly large under high invocation frequencies. Since the implementation of a USDT via `libbpf-rs` was not successful, a uprobe must be used. However, as the synchronization is performed only once and the overhead plays only a minor role, this choice is acceptable.

The following section discusses the technical implementation of the synchronization. This process is illustrated in Figure 4.10. As can be seen, a uretprobe is used for time synchronization. Two timelines are depicted, one for user space and one for kernel space. The user-space timeline starts at time t_{u0}, while the kernel-space timeline begins at time t_{k0}. As previously mentioned, the problem is that $t_{u0} \neq t_{k0}$. Therefore, it is desirable to establish a common reference point on both timelines from which times can be measured relative to each other. This is achieved by the uretprobe.

As discussed in Section 4.3, the implementation of a uretprobe requires a uprobe. The uprobe is attached to a user-space function and is triggered upon entering this function. In Figure 4.10, the function is entered at t_{u1} in user space, and via the `int3` breakpoint, control is passed to the uprobe in kernel space, which receives control at t_{k1}. It holds that $t_{k1} > t_{u1}$[5], since the context switch to the uprobe requires time. This interval is highlighted in blue in Figure 4.10 and corresponds to $t_{k1} - t_{u1}$, which can not be determined. As described in Section 4.3, the uprobe only manipulates the return address in order to prepare the trampoline for the uretprobe. Thus, the uprobe terminates at t_{k2} and returns control to the user-space function. The duration of the uprobe is marked in yellow in Figure 4.10 and has the value $t_{k2} - t_{k1}$.

At t_{u2}, the function in user space regains control. It then sets a timer using the system call `clock_gettime()`. This syscall again requires a transition into kernel space. The duration of this transition is shown in green in Figure 4.10. The syscall executes in kernel space starting at t_{k3} and completes at t_{k4}. At t_{u3}, the user-space function regains control, and the timer is set. This point is marked by a orange dot in the figure and represents the relative moment from which measurements can be taken. Since the function returns at this point, the uretprobe is triggered.

The uretprobe again switches to kernel space. This transition is slightly longer than the syscall transition and is indicated in blue, analogous to the uprobe. At t_{k5}, the uretprobe receives control and, using the function `bpf_ktime_get_ns()`, measures the system time in nanoseconds. Thus, at t_{k6}, the relative program start time in kernel space is set. At t_{k7}, the uretprobe terminates and returns control to the user-space function, which resumes execution at t_{u4}.

At this point, reference values have been established in both kernel and user space that can be used for future measurements. When a tracepoint such as `consume_skb` is triggered, a timestamp can be obtained in kernel space that can be correlated with the corresponding user-space timestamp. As shown in Figure 4.10, at t_{k8} the tracepoint is triggered and at t_{k9} the time is measured using `bpf_ktime_get_ns()`. The tracepoint is exited at t_{k10}, and control flows through the network layers back to the application in user space, as illustrated in Figure 4.9. The application regains control at t_{u6} and measures the time at t_{u7} using `Instant::now()`. The time difference d_u between the program start at t_{u3} and the measurement at t_{u7} can thus be related to the time difference d_k, which is defined as the difference between the kernel-space program start t_{k6} and the tracepoint entry t_{k9}. Consequently, the time needed by t_4 and t_8 in

5 Like stated before, these timers can not be compared, because at this point no shared time basis is available. But if they were comparable, the described statement would apply

Figure 4.9 can be determined as $d_u - d_k$.

It is difficult to determine the delay between the user-space time t_{u3} and the kernel-space time t_{k6}. This delay reflects the accuracy of the synchronization between user space and kernel space. Zheng et al. report in [76] that the uretprobe introduces an overall overhead of approximately 4 μs. However, since the delay does not account for the entire duration of the uretprobe, it is expected to be significantly smaller.

To estimate this delay, measurements can be performed in user space using `Instant::now()`. From this, Equation 4.3 is derived for the delay d_{ku} between user space and kernel space. Accordingly, the duration $t_{uretprobe}$ of the uretprobe can be measured as $t_{u4} - t_{u1}$. By replacing the uretprobe with a uprobe, the execution time of the uprobe can be determined. This corresponds to the duration $t_{uprobe} = t_{u3} - t_{u1}$. It must be noted that the uprobe returns immediately and does not, unlike the uretprobe shown in Figure 4.10, include the timing and ring-buffer logic, as this would distort the measurement results. This logic is shown later in this chapter.

$$d_{ku} = \frac{1}{2}[(t_{u4} - t_{u1}) - (t_{u3} - t_{u1})] \tag{4.3}$$

In this way, the measurement results presented in Table 4.2 were obtained. For each case, 100 measurements were performed in order to determine the maximum and average values of latency and jitter. The table reports both the maximum and average values for d_{ku} as well as for jitter, in addition to the average durations of $t_{uretprobe}$ and t_{uprobe}.

Furthermore, these values are provided for two different timers in Rust. Specifically, the measurements were conducted using the timer `SystemTime::now()`, which, as explained earlier, is said to be unsuitableand and the timer `Instant::now()`. In addition, the experiments were carried out once with real-time priority enabled and once without real-time priority.

Table 4.2 Comparison of the timesynchronization between User- and Kernelspace with different timer and priority

Kind of Timer	**SystemTime**	**SystemTime and real time priority**	**Instant**	**Instant and real time priority**
$t_{uretprobe}$	6580 ns	5920 ns	5120 ns	4500 ns
t_{uprobe}	3940 ns	3560 ns	3390 ns	3310 ns
d_{ku-max}	3120 ns	2740 ns	1950 ns	1440 ns
d_{ku-avg}	2640 ns	2360 ns	1630 ns	1190 ns
Max Jitter	470 ns	440 ns	360 ns	310 ns
Avg Jitter	380 ns	350 ns	320 ns	280 ns

The results reveal two clear tendencies. First, the function `Instant::now()` is faster than `SystemTime::now()` both on average and in the maximum case and it also exhibits lower jitter. Second, the measurements with real-time priority consistently outperform those without real-time priority. Both tendencies were expected, since it was already shown in [112] that `SystemTime::now()` is less accurate than `Instant::now()`. Real-time priority, as described in Section 2.7.2, allows the scheduler to prioritize execution, which generally results in lower latency and jitter. Since both timers, `SystemTime::now()` and `Instant::now()`, require a transition into kernel space, it was to be expected that prioritization by the scheduler would accelerate this process.

Another interesting observation concerns the duration of the uretprobe and the uprobe. Zheng et al. report in [76, p. 7] that the uretprobe introduces an overhead of about 4 μs, while the uprobe introduces an overhead of about 3 μs. In the measurement results shown in Table 4.2, the times for both probes are higher. This can be explained by the fact that this work uses a

Raspberry Pi 5, which presumably has lower computational power compared to the CPU used by Zheng et al. Although [76] does not specify the exact CPU, it is reasonable to assume that a more powerful processor was employed, as the paper was published in 2023 and the CPU of the Raspberry Pi 5 cannot match the performance of a desktop CPU from this time. Furthermore, additional overhead may be introduced by performing the measurements with Rust user-space functions. Nevertheless, the results do not deviate substantially from the values reported in [76], which supports the conclusion that the values obtained for d_{ku} are reliable.

As a final conclusion, the measurements demonstrate that synchronization between user space and kernel space can be achieved with the uretprobe at an accuracy of approximately 1–1.5 μs. Moreover, the results confirm the observation from [112] that `Instant::now()` provides more accurate timing than `SystemTime::now()`. Finally, it can be concluded that synchronization should be performed after assigning real-time priority in order to maximize accuracy.

Listing 6 shows the function to which the uretprobe is attached. This function sets the variable `USER_ZERO` to the current timestamp returned by `Instant::now()`, thereby establishing the time t_{u3} shown in Figure 4.10.

It can be observed that the function is annotated with the attribute `#[no_mangle]` and declared with the keyword `extern "C"`. The attribute `#[no_mangle]` is required to prevent Rust from modifying the function name during compilation, as described in [113]. The keyword `extern "C"` specifies that the C Application Binary Interface (ABI) is used for this function, as discussed in [114]. This is necessary to ensure that C code[6] can correctly invoke it, particularly with respect to calling conventions and parameter passing.

```
1 #[no_mangle]
2 pub extern "C" fn measure_instant() {
3     let mut time = USER_ZERO.lock().unwrap();
4     *time = Instant::now();
5 }
```

Listing 6 Function in Rust to which the uretprobe is attached

This is relevant to ensure that the uretprobe can be attached to the function. Listing 7 shows the structure of the function to which the uretprobe is attached. As described in [115], the uretprobe is created using `SEC("uretprobe/path to file:function-name")`. Therefore, it is crucial that the function name remains unchanged and that the function can be invoked by C code.

In the execution of the function, a timestamp is generated first in order to keep d_{ku} as small as possible. The timestamp represents the point in time t_{k6} in Figure 4.10. Subsequently, memory is reserved in the ring buffer using the function `bpf_ringbuf_reserve`, after which the event is submitted via `bpf_ringbuf_submit` into it, in order get consumed by the user space. After the message is submitted into the rinbuffer the user sace gets a notification that there is an message to collect, like described in [116]. The event consists of a type identifier `event_type`, a data structure containing the parameters `msg_type` and `seq` and a timestamp. For the uretprobe, only the event type and the timestamp are relevant. The event type allows the user-space program to determine how the event should be processed. For this reason, a unique event type is assigned to each BPF function[7]. For the BPF function of the uretprobe, this type is set to 0. In addition, the generated timestamp is passed in the field `timestamp`.

6 Since BPF is based on C code, the C ABI is required so that a uprobe or uretprobe can be attached to the function.

7 Each event source can be associated with a different BPF function. In this work, a distinct BPF function is attached to the uretprobe as well as to the three tracepoints discussed earlier in this chapter.

```
SEC("uretprobe//code/client_udp/target/debug/client_udp:measure_instant")
int trace_measure_instant(struct pt_regs *ctx) {
    __u64 timestamp = bpf_ktime_get_ns();
    struct Event *e = bpf_ringbuf_reserve(&events, sizeof(*e), 0);
    if (!e) {
        return 0;
    }
        e->event_type = 0;
    e->data.msg_type = 0;
    e->data.seq = 0;
    e->timestamp = timestamp;

    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

Listing 7 Handling uretprobe in BPF

Having explained how user-space and kernel-space synchronization is achieved, the following section describes how client-server synchronization is implemented. This synchronization is carried out in three phases, namely NTP, PTP and Check. As previously mentioned, these phases are named after time-synchronization protocols. However, only their functionality is implemented, not the full protocol specification.

NTP was developed in 1991 by Mills, like he wrote in [117]. At the time of this work, it has been in operation for more than 30 years. As described by Chefrour in [118, p. 31], NTP achieves an accuracy ranging from tens of microseconds to several hundred milliseconds, depending on the network conditions. As a successor, and particularly with a focus on improved performance, PTP was developed in 2002, as reported in [119]. This protocol, however, was primarily designed for Local Area Network (LAN) environments. According to Chefrour [118, p. 31], PTP can achieve accuracies between 10 and 100 ns, but only when implemented in hardware, which requires specialized network chips.

Both NTP and PTP are, as highlighted in [118, p. 29], significant milestones and key protocols in network synchronization. For this reason, these names are used here to denote the phases of coarse synchronization and fine synchronization, respectively. Both protocols rely on the assumption of a symmetric network. In other words, it is assumed that the transmission time from x to y equals the transmission time from y to x. The synchronization approach used in this work is based on the same principle.

The principle of synchronization is illustrated in Figure 4.11. This figure shows both a flow chart of the three states and their respective iterations as a sequence diagram.

The time synchronization begins with the NTP phase, which is represented by the NTP state in Figure 4.8. A predefined number of repetitions is executed until the RTT falls below the specified requirement RTT_{Req}. An NTP repetition is illustrated in Figure 4.11. In this process, the server first synchronizes its user space with kernel space, as described earlier. It then sends a message of type NTP to the client, as shown in Listing 5. Once the client receives the NTP message, it also synchronizes its user space and kernel space and responds with a message of type NTP. The server waits for the client's response and can thus determine RTT_{NTP}. This ensures that the server and client are roughly synchronized, with an error bounded by $\frac{\mathrm{RTT}_{NTP}}{2}$. This assumption holds only under symmetric network conditions, which Wi-Fi does not reliably provide. Nevertheless, it guarantees that the time deviation between client and server is less than RTT_{NTP}. Therefore, it is advantageous to minimize the RTT, which is why the NTP phase enforces the requirement $\mathrm{RTT}_{NTP} < RTT_{Req}$. If this condition is not satisfied, the NTP repetition is repeated, otherwise, the state changes to PTP.

The resulting RTT_{NTP} serves as the basis for fine synchronization. The procedure of the

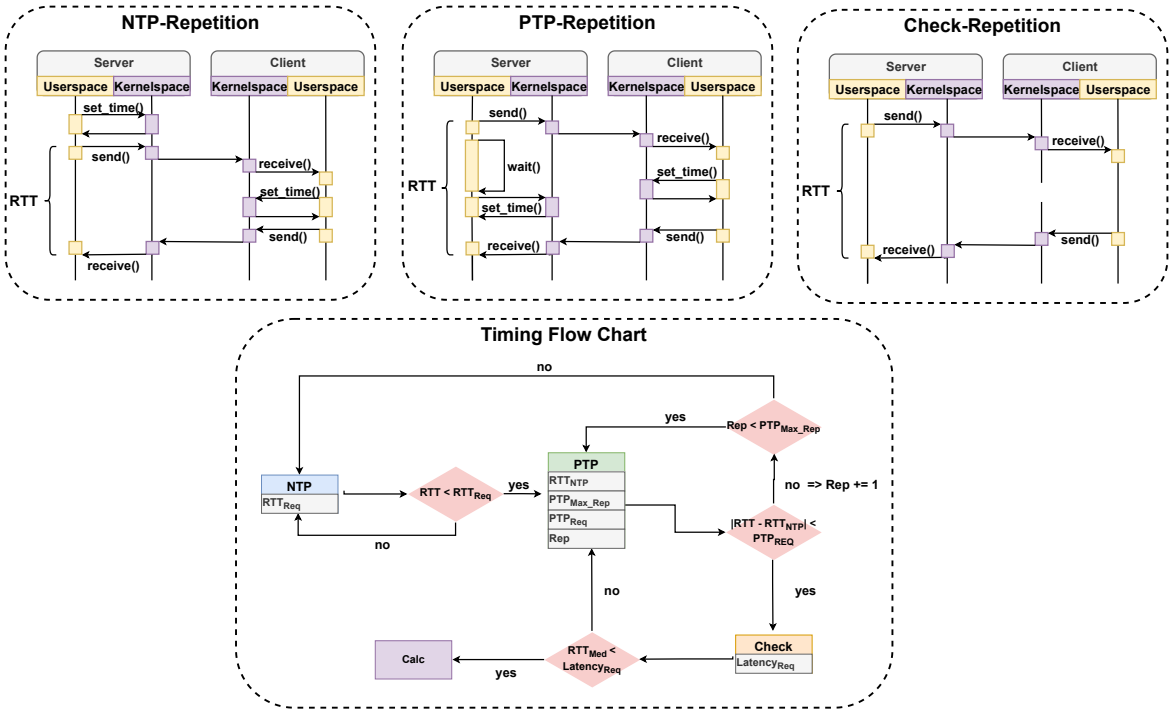

Figure 4.11 Presentation of a Flow Chart of the synchronization between Client and Server

PTP phase is illustrated as PTP repetition in Figure 4.11. In this phase, the server again sends a message to the client, this time of type PTP. Unlike in the NTP phase, the server does not synchronize its user and kernel space before sending the message, but rather after a delay. Specifically, it waits for $\frac{\text{RTT}_{NTP}}{2}$ after sending the message before performing synchronization. The client behaves identically to the NTP phase, except that it returns a packet of type PTP to the server. The server measures RTT_{PTP} and checks whether $|\text{RTT}_{NTP} - \text{RTT}_{PTP}| < \text{PTP}_{Req}$ holds. If the condition is satisfied, the process transitions to the Check state, otherwise, the repetition is executed again. This approach ensures that the server can predict the client's reception time of the packet, thereby improving synchronization accuracy. In the case of a symmetric network, the maximum synchronization error is PTP_{Req}. However, RTT_{NTP} may not always be reproducible, since it could have been obtained under exceptionally favorable conditions that cannot be recreated. For this reason, the PTP phase is limited to a maximum number of repetitions. If this limit is exceeded, RTT_{NTP} must be redetermined, and the system transitions back to the NTP state.

The final phase of the time-synchronization process is the Check state. This step is necessary because Wi-Fi is not always symmetric. In this phase, the server sends a message of type Check to the client while recording a timestamp t_{ss}. The client records the two timestamps t_{cr} upon receiving the message and t_{cs} upon sending the response. It then returns these values to the server in a message of type Check. Upon reception, the server records another timestamp t_{sr}. From these values, the one-way delay from server to client is computed as $t_{s-c} = t_{cr} - t_{ss}$, and the delay from client to server as $t_{c-s} = t_{sr} - t_{cs}$. If the Wi-Fi connection is symmetric and the synchronization is correct, then $t_d = t_{s-c} - t_{c-s} = 0$. In the Check phase, this procedure is repeated several thousand times, after which the median M_{Check} of the t_d values is calculated. If $M_{Check} < Latency_{Req}$, real-time computation is initiated, otherwise, the system returns to the PTP state. Although Wi-Fi tends to be symmetric on average over longer periods, this is not guaranteed for individual packets. Therefore, the median[8] is used to determine whether synchronization is sufficiently accurate. While deviations still occur, under the assumption of

8 In this case, the median is used because it is less sensitive to outliers and thus provides more stable results.

long-term Wi-Fi symmetry, the maximum synchronization error between client and server is bounded by $Latency_{Req}$. In this work, a threshold of 10 μs is applied.

Following the Check state is the Calc state. As shown in Figure 4.8, this is the only state with a real-time requirement. The implementation of this state has already been explained in Figures 4.7 and 4.9. The number of repetitions depends, as illustrated in Figure 4.7, on the number of measurement points. The following section discusses how data can be collected using BPF and how it is processed in Rust.

The code used to read a tracepoint is nearly identical for all tracepoints. As an example, the procedure for the tracepoint `net_dev_xmit` is explained. The code of the function executed when the tracepoint triggers is shown in Listing 11. Similar to the uprobe, the function is attached to the tracepoint using `SEC("tracepoint/net/net_dev_xmit")`. As shown in lines 3–5, the Process Id (PID) must first be determined using the function `bpf_get_current_pid_tgid()`, and the process name must be retrieved using `bpf_get_current_comm()`. Subsequently, the process name (`comm`) is verified to ensure that the message originates from the correct process.

In lines 8–13, the addresses of `head` and `mac_header` are extracted. These addresses are obtained from the `sk_buff` struct, which, as described in [120], represents a network message. To extract these fields, the function shown in Listing 8 is used. This function determines the address of a struct member using the `offsetof()` function.

```
1  #define member_read(destination, source_struct, source_member)
2  do{
3    bpf_probe_read(
4      destination,
5      sizeof(source_struct->source_member),
6      ((char*)source_struct) + offsetof(typeof(*source_struct), source_member)
7    );
8  } while(0)
```

Listing 8 Function to read members out of a struct

Using `head` and `mac_header`, the location of the MAC header can be determined. For precise identification of the network packet, the IP header is also required. This header follows directly after the MAC header and can therefore be computed as shown in line 14 of Listing 11. Subsequently, it is checked whether the packet is a TCP packet or not. If not, the function is aborted. Additionally, lines 22–28 of Listing 11 extract the last octet of the source and destination IP addresses, i.e., the `yz` part from `192.168.wx.yz`. It is then verified whether the server IP ends with 1 and the client IP ends with 43. If this condition is not met, the function is aborted, as the packet is either destined for the wrong recipient or sent by the wrong sender.

The TCP header is also required to access the payload. The TCP header follows immediately after the IP header, and its length must be known. This length is computed in line 29. For this purpose, the IP header length (`ihl`) is used and multiplied by 4. This is necessary because `ihl` specifies the header length in 32-bit words, as described in the standard [121, p. 11]. In other words, if `ihl` returns 5, the IP header consists of 5×32 bits, i.e., 5×4 bytes.

In line 36, the offset for the payload is determined. For this calculation, the TCP header length is required. This length can be computed using the data offset, which again provides the size in words, as described in the standard [122, p. 16].

Finally, the message specified in Rust is loaded. A check is performed again to ensure that the message type is not out of bounds. If the check passes, the event is assembled and submitted to the ring buffer. This procedure has already been illustrated in Listing 7.

The data sent via the ring buffer are received in user space by Rust and stored as events in separate queues. In other words, a dedicated queue is provided for each tracepoint event. As shown in Listing 7, the event also contains the PID. This identifier is used to verify whether the

event in the ring buffer originates from BPF. Since the ring buffer is not exclusively assigned to BPF but represents a system-wide resource, this verification is necessary. Subsequently, the `net_dev_xmit` event is written into the send queue in user space. The same procedure applies to the other events.

```
1  fn calculation_phase(){
2      let start_time = Instant::now();
3      let theta = 2.0 * PI * (i as f64) / (NUM_POINTS as f64);
4      let x = RADIUS * theta.cos();
5
6      let encoded_msg = encode_message(MessageType::Calc, i, theta, RADIUS, 0)?;
7      stream.write_all(&encoded_msg);
8
9      let queue_event = wait_for_event(queue);
10     let server_queue_time = queue_event.timestamp;
11
12     let send_event = wait_for_event(send);
13     let server_send_time = send_event.timestamp;
14
15      match stream.read(&mut buffer) {
16          let end_time = Instant::now();
17          let msg: Message = *bytemuck::from_bytes::<Message>(&buffer);
18
19          let receive_event = wait_for_event(receive);
20          let server_receive_time = receive_event.timestamp;
21
22          calc_monitoring(
23              server_queue_time,
24              ...,
25              msg.client_queue_time,
26              ...);
27
28          if end_time.duration_since(start_time) <= TIMEOUT_NS as u128 {
29              y
30          } else {
31              y - penalty
32          };
33      }
34 }
```

Listing 9 Presentation of Pseudo-Code of the calculation function

Listing 9 illustrates the flow of the `calculation` function on the server as pseudo-Rust code. As previously mentioned, the start time is first recorded and the x-coordinate is computed, before the angle for which the y-coordinate is to be calculated is sent to the client. This process is shown in lines 2–7. Subsequently, the tracepoint events for `net_dev_xmit` and `net_dev_queue` are awaited. This is realized through the function `wait_for_event()`, which takes as a parameter the queue in which the corresponding event is collected. The function returns the event, from which the timestamp of its occurrence can be extracted via the `timestamp` attribute.

Afterward, the response from the client is awaited, and upon its arrival the end time is recorded. The message contains the y-value and the client's timing data. These, combined with the server's timing data, are used as shown in lines 22–26 to calculate the monitoring times described in Figure 4.9. Finally, it is checked whether the interval between start and end time exceeds the specified time limit, thereby violating the real-time requirement. If this is the case, the previously mentioned penalty is applied to y.

After this state, only two states remain, as shown in Figure 4.8. The first is the *Error* state, which is also a terminal state. It can be reached from any synchronization state as well as from the *Calc* state. This state is entered under one of the following conditions:

- **Client timeout:** If the client terminates the TCP connection or does not respond in the case of User Datagram Protocol (UDP).
- **Incomplete dataset:** If the data collected by the server[9] are incomplete. In other words, if not all points contain data or if certain data for specific points are missing.

The error state terminates both the server and the client and informs the test automation system that the test must be repeated.

The second state, which has not yet been described, is the *Save* state. In this state, the data collected during *Calculation* are written to a file, where they can later be retrieved and visualized. This state concludes in the *End* state, which marks the successful completion of the test run.

This chapter has described the application, its functionality, and its implementation. The following section will address how the application is embedded into a test run and how the test executions are managed.

4.4.3 Test Automatization

The organization and automation of the test executions are implemented by means of a test suite. As described in Chapter 4.2.3, this suite is implemented in Python. There is one test suite for the client and one for the server. However, these are largely similar in their structure and functionality.

Figure 4.12 illustrates the workflow of the server test suite. Based on this figure, the workflow of the test suite will be explained in the following. The workflow of the client test suite is also explained on this basis. In the figure the test suite for the server is shown, it will be explained, when the client test suite differs from it.

The test suite begins by compiling the Rust code in order to ensure that the latest version is used. Subsequently, in the server test suite, the iperf3 server is started in a separate thread. This step does not exist in the client test suite. On the client side, the iperf3 client is not launched by the test suite itself, but rather by the application once the check state has been successfully finished.

In both test suites, the next step is the selection of the test run. For this purpose, the status file is first checked, which contains a digit specifying which test run should be executed next. Afterwards, the configuration file is opened, which contains the different configurations. Each configuration corresponds to one line in this file, specifying the Wi-Fi standard, frequency, bandwidth, Quality of Service (QoS) status, protocol and AP configuration name. For instance, the line `4 2.4 20 0 udp wifi4` specifies that the Wi-Fi connection should use standard 4, a frequency of 2.4 GHz, and a bandwidth of 20 MHz. Moreover, no QoS parameters should be applied, the application should use UDP, and the corresponding hostapd configuration for the Wi-Fi parameters is included in the config file `wifi4`. The last parameter is not relevant for the client test suite since it has not to start an AP with hostapd.

The next step on the server side is to start the AP with the respective hostapd configuration. The client test suite does not include this step. Instead the client establishes a connection to the server using wpa-supplicant. The hostapd configurations are explained in more detail in Chapter 4.4.4.

In the subsequent step, both test suites set or reset the QoS parameters. Afterwards the application, either the server or the client, is started in a thread. In the next step, the test executions proceed, which differ between server and client. On the server side, only the application

9 The dataset consists of the monitoring data and the circle data, i.e., the x and y values.

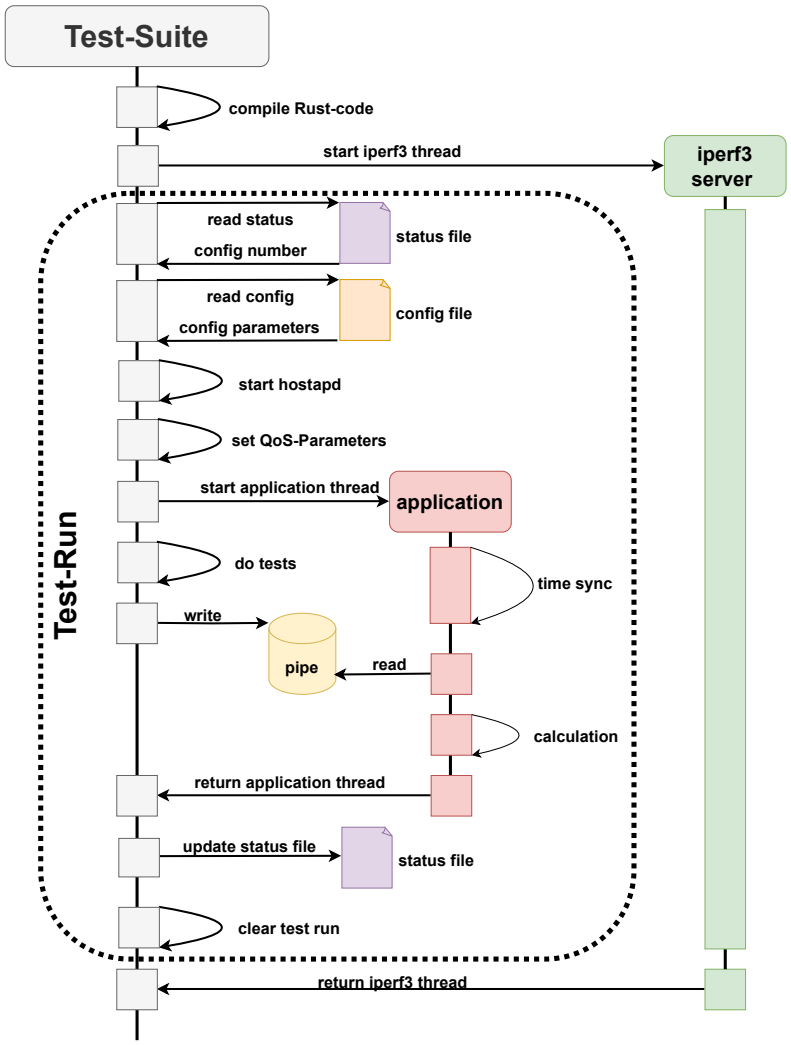

Figure 4.12 Presentation of sequence diagram of the test-suite of the server

priority is checked to ensure that it is set to 139, i.e., real-time priority, while the iperf process runs at a lower priority. This is achieved by executing the shell command `ps -eLo comm,pri | grep server/iperf` in Python and verifying that the server process runs at priority 139, whereas the iperf process does not. As described in [123], this command lists all processes including threads and outputs only the name and the priority. On the client side, the priorities are checked in the same way. Additionally, it is verified that Wi-Fi version, bandwidth, and frequency match the values specified in the configuration. In other words, the client test suite checks that the server's AP configuration is correct and actually usable[10].

The bandwidth can be determined using the command `iw dev IFACE info`. The value is found after the parameter `width:` and can be extracted with the Regular Expression (RegEx) $re.search(r'width:\s*(\d+)\s*(?=MHz)', output)$. The frequency and Wi-Fi standard can be determined with the command `iw dev IFACE link`, as described in [124]. The frequency is found after the argument `freq` and can be extracted using the RegEx $re.search(r'freq:$

10The fact that an AP provides certain features, such as Wi-Fi standard 6, does not automatically imply that these features are active in the client connection. For example, if the client connects to the AP with an incompatible Wi-Fi Protected Access (WPA) configuration, the Wi-Fi standard may be downgraded. Therefore, it is necessary to perform the verification on the client side to guarantee that all configurations are applied correctly.

$\backslash s * (\backslash d + \backslash .\backslash d+)', output)$. For determining the Wi-Fi standard, the full output is analyzed for the flags High Throughput (HT), Very High Throughput (VHT) and High Efficiency (HE). HT is mandatory for Wi-Fi 4, VHT for Wi-Fi 5 and HE for Wi-Fi 6. If none of these flags is present in the output, the standard is older than Wi-Fi 4.

Once the tests are completed, the string `ready` is written into a pipe, which the Rust application reads to signal that the tests have finished. This ensures that the tests do not interfere with the real-time capabilities of the application. In the check state, the application waits for the `ready` signal from the pipe. Even if the tests fail, the application is executed to completion, but the test results are logged, allowing one to identify which properties were not fulfilled. The tests are implemented using `py-unittest`. This ensures that the desired configurations are validated and enforced.

After the application finishes, the status file is updated. If the application terminates successfully in the state `End`, the digit in the status file is incremented by one. If, however, the application terminates in the state `Error`, the digit remains unchanged. In this way, the current test run will be repeated. The next step is to clean up the entire configuration. This means that QoS parameters are reset if necessary, and the AP is terminated. On the client side, the WPA configuration is reset.

Finally, the iperf3 server thread is terminated. Figure 4.12 illustrates a test run. This represents the section that is repeated at least once for each configuration entry in the configuration file. Consequently, the Rust project is compiled only once for both client and server. On the server side, the iperf3 server is also started only once, since it remains persistent. By contrast, the iperf3 client reconnects for each test run, since it may require a different configuration. This process is explained in more detail in Chapter 5.

Having now described the organization of the test runs, the following section addresses the implementation of the APs using hostapd.

4.4.4 Access point configuration

5 Wi-Fi Modifications

5.1 Quality of Service

5.2 Change of Wi-Fi Standards

5.3 Frequency

5.4 Bandwidth

6 Evaluation

6.1 Test procedure

6.2 Results of low disturbance test

6.3 Results of normal use test

6.4 Results of high disturbance test

6.5 Discussion

7 Conclusion

A First Chapter of the Appendix

```rust
include!("bpf/monitore.skel.rs");
...
set_rt_priority(99);
...
let open_skel = MonitoreSkelBuilder::default().open();
let mut skel = open_skel?.load()?;
skel.attach()?;

let maps = skel.maps();

let mut ringbuf_builder = RingBufferBuilder::new();
ringbuf_builder.add(maps.events(), move data: &[u8] {
    // Callback function
});

let _handle = thread::spawn((move {
    while true {
        ringbuf.poll(Duration::from_millis(100)).unwrap();
    }
});

let listener = TcpListener::bind("192.168.1.1:8080")?;

for stream in listener.incoming(){
    match stream {
        Ok(stream) => {
            let handle = thread::spawn((move  {
                let context = SetupContext {
                        ...
                };
                _ = run_state_machine(context);
            });
            handle.join();
        }
    }
}
```

Listing 10 Parts of the setup state of the server

```
SEC("tracepoint/net/net_dev_xmit")
int handle_net_dev_xmit(struct trace_event_raw_net_dev_xmit *ctx) {
    int pid = bpf_get_current_pid_tgid() >> 32;
    char comm[16] = {};
    bpf_get_current_comm(&comm, sizeof(comm));

    if (__builtin_strcmp(comm, "server") == 0) {
        struct sk_buff *skb = (struct sk_buff *)ctx->skbaddr;
            char *head;
        u16 mac_header;

            member_read(&head, skb, head);
            member_read(&mac_header, skb, mac_header);

            char* ip_header_address = head + mac_header + MAC_HEADER_SIZE;
            struct iphdr iph;
        bpf_probe_read(&iph, sizeof(iph), ip_header_address);
        if (iph.protocol != IPPROTO_TCP)
            return 0;


            u32 src_ip = __builtin_bswap32(iph.saddr);
            u8 src_lat_ip = src_ip & 0xff;

            u32 dst_ip = __builtin_bswap32(iph.daddr);
            u8 dst_last_ip = dst_ip & 0xff;

        if(src_lat_ip == 1 && dst_last_ip == 43){
                u8 ip_header_len = iph.ihl * 4;


                    char *tcp_header = ip_header_address + ip_header_len;

                    struct tcphdr tcph = {};
                    bpf_probe_read(&tcph, sizeof(tcph), tcp_header);
                    u8 tcp_header_len = tcph.doff * 4;

                char *payload = tcp_header + tcp_header_len;

                        struct Message msg = {};
                        bpf_probe_read(&msg, sizeof(msg), payload);
                    if (msg.msg_type < 0  msg.msg_type > 5) return 0;
                        ...

                        bpf_ringbuf_submit(event, 0);
                }
    }

    return 0;
}
```

Listing 11 Handling tracepoint in BPF

Bibliography

[1] "Linux kernel network stack and architecture." [Online]. Available: https://thelinuxchannel.org/2024/07/linux-kernel-network-stack-and-architecture/

[2] G. C. Buttazzo, *Hard real-time computing systems: Predictable scheduling algorithms and applications*. Springer, 2024.

[3] M. P. Rooney, N. Rao, N. Liebers, A. S. Leger, and S. J. Matthews, "A comparative study of programming languages for a real-time smart grid application," in *2023 IEEE Green Energy and Smart Systems Conference (IGESSC)*, 2023, pp. 1–6.

[4] Mediatek, "mt76," Nov 2014. [Online]. Available: https://github.com/openwrt/mt76

[5] M. Mossige, P. Sampath, and R. Rao, "Evaluation of linux rt-preempt for embedded industrial devices for automation and power technologies-a case study," 01 2007. [Online]. Available: https://www.osadl.org/fileadmin/events/rtlws-2007/Sampath.pdf

[6] M. M. ARSHIABANU, "Rt linux based priority scheduling using raspberry pi," *International Journal of Scientific Engineering and Technology Research*, vol. 3, no. 29, p. 5921–5924, Oct 2014. [Online]. Available: https://ijsetr.com/uploads/624531IJSETR2461-1042.pdf

[7] W. Dewit, A. Paolillo, and J. Goossens, "A preliminary assessment of the real-time capabilities of real-time linux on raspberry pi 5," The 18th annual workshop on Operating Systems Platforms for Embedded Real-Time applications, WorkingPaper, Jul. 2024.

[8] S. Arthur and N. Mc Guire, "Assessment of the realtime preemption patches (rt-preempt) and their impact on the general purpose performance of the system," 01 2007. [Online]. Available: https://www.osadl.org/fileadmin/events/rtlws-2007/Siro.pdf

[9] A. Carvalho, C. Machado, and F. Moraes, "Raspberry pi performance analysis in real-time applications with the rt-preempt patch," in *2019 Latin American Robotics Symposium (LARS), 2019 Brazilian Symposium on Robotics (SBR) and 2019 Workshop on Robotics in Education (WRE)*, 2019, pp. 162–167.

[10] W. Betz, M. Cereia, and I. C. Bertolotti, "Experimental evaluation of the linux rt patch for real-time applications," in *2009 IEEE Conference on Emerging Technologies and Factory Automation*, 2009, pp. 1–4.

[11] C.-F. Yang and Y. Shinjo, "Compounded real-time operating systems for rich real-time applications," *IEEE Access*, vol. 13, pp. 26 079–26 104, 2025.

[12] M. D. Marieska, P. G. Hariyanto, M. F. Fauzan, A. I. Kistijantoro, and A. Manaf, "On performance of kernel based and embedded real-time operating system: Benchmarking and analysis," in *2011 International Conference on Advanced Computer Science and Information Systems*, 2011, pp. 401–406.

[13] S. Rostedt and D. Hart, "Internals of the rt patch," pp. 161 – 172, 01 2007.

[14] F. Vasquez and C. Simmonds, 2021.

[15] A. P. Navik and D. Muthuswamy, "Dual band wlan gateway solutions in yocto linux for iot platforms," in *2017 International Conference on Internet of Things for the Global Community (IoTGC)*, 2017, pp. 1–3.

[16] O. Salvador and D. Angolini, 2023.

[17] E. Anderson, "Build your own wireless access point," pp. 33 – 40, 2004. [Online]. Available: https://www.kernel.org/doc/ols/2004/ols2004v1-pages-33-40.pdf

[18] "hostapd linux documentation page." [Online]. Available: https://wireless.docs.kernel.org/en/latest/en/users/documentation/hostapd.html#using-your-distribution-s-hostapd

[19] "wpa supplicant.conf - linux man page." [Online]. Available: https://linux.die.net/man/5/wpa_supplicant.conf

[20] J. Dugan, S. Elliott, B. A. Mah, J. Poskanzer, and K. Prabhu, "iperf - the ultimate speed test tool for tcp, udp and sctp." [Online]. Available: https://iperf.fr

[21] P. Vale and F. Pereira, "Automatic python code generation for embedded/cyber-physical systems," in *2023 7th International Young Engineers Forum (YEF-ECE)*, 2023, pp. 49–55.

[22] J. Sommer, T. Gutierrez Rojo, A. Lund, H. I. E. Abdelmaksoud, and D. Lüdtke, "Viability of rust for avionics software development – current status and way forward," in *7th Workshop on Avionics Systems and Software Engineering*, ser. SE 2025 - Companion Proceedings. Gesellschaft für Informatik, Februar 2025. [Online]. Available: https://elib.dlr.de/212971/

[23] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek, "Benefits and drawbacks of adopting a secure programming language: rust as a case study," in *Proceedings of the Seventeenth USENIX Conference on Usable Privacy and Security*, ser. SOUPS'21. USA: USENIX Association, 2021.

[24] S. Panter and N. Eisty, "Rusty linux: Advances in rust for linux kernel development," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, ser. ESEM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 496–502. [Online]. Available: https://doi.org/10.1145/3674805.3690756

[25] L. Seidel and J. Beier, "Bringing rust to safety-critical systems in space," 2024. [Online]. Available: https://arxiv.org/abs/2405.18135

[26] R. F. Team, "Announcing the safety-critical rust consortium," 2024.

[27] W. Youn, S. Hong, K. Oh, and O. Ahn, "Software certification of safety-critical avionic systems: Do-178c and its impacts," *Aerospace and Electronic Systems Magazine, IEEE*, vol. 30, pp. 4–13, 04 2015.

[28] B. Gregg, *BPF performance tools: Linux system and application observability.* Addison-Wesley: Pearson Education, 2020.

[29] P. G. Kannan, S. M. Gupta, D. Behl, E. Raichstein, and J. Takvorian, "Designing a lightweight network observability agent for cloud applications," 2024. [Online]. Available: https://pam2024.cs.northwestern.edu/pdfs/paper-75.pdf

[30] C. Liu, Z. Cai, B. Wang, Z. Tang, and J. Liu, "A protocol-independent container network observability analysis system based on ebpf," in *2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS)*, 2020, pp. 697–702.

[31] "Releases." [Online]. Available: https://wiki.yoctoproject.org/wiki/Releases

[32] "Raspberry pi." [Online]. Available: https://github.com/raspberrypi/firmware/tree/master/boot/overlays

[33] "cargo-bitbake." [Online]. Available: https://crates.io/crates/cargo-bitbake/0.2.0/dependencies

[34] "cargo-vendor." [Online]. Available: https://doc.rust-lang.org/cargo/commands/cargo-vendor.html

[35] S. McCanne and V. Jacobson, "The BSD packet filter: A new architecture for user-level packet capture," in *USENIX Winter 1993 Conference (USENIX Winter 1993 Conference)*. San Diego, CA: USENIX Association, Jan. 1993. [Online]. Available: https://www.usenix.org/conference/usenix-winter-1993-conference/bsd-packet-filter-new-architecture-user-level-packet

[36] "Extended bpf." [Online]. Available: https://lkml.org/lkml/2013/9/30/627

[37] B. Gregg, *Systems performance, 2nd edition*. Pearson, 2020.

[38] B. Gbadamosi, L. Leonardi, T. Pulls, T. Høiland-Jørgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunström, "The ebpf runtime in the linux kernel," 2024. [Online]. Available: https://arxiv.org/abs/2410.00026

[39] "bpf(2) — linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man2/bpf.2.html

[40] J. Van Geffen, L. Nelson, I. Dillig, X. Wang, and E. Torlak, "Synthesizing jit compilers for in-kernel dsls," in *Computer Aided Verification*, S. K. Lahiri and C. Wang, Eds. Cham: Springer International Publishing, 2020, pp. 564–586.

[41] E. Bugnion, J. Nieh, and D. Tsafrir, *Definitions*. Cham: Springer International Publishing, 2017, pp. 1–14. [Online]. Available: https://doi.org/10.1007/978-3-031-01753-7_1

[42] M. Fleming, "A thorough introduction to ebpf." [Online]. Available: https://lwn.net/Articles/740157/

[43] T. Zanussi, "relayfs, a high-speed data relay filesystem." [Online]. Available: https://lwn.net/Articles/25791/

[44] R. W. Wisniewski, "relayfs : An efficient unified approach for transmitting data from kernel to user space," 2003. [Online]. Available: https://api.semanticscholar.org/CorpusID:15659650

[45] P.-M. Fournier, M. Desnoyers, and M. R. Dagenais, *Combined Tracing of the Kernel and Applications with LTTng*, 2009, pp. 87–94. [Online]. Available: https://www.kernel.org/doc/ols/2009/ols2009-pages-87-94.pdf

[46] D. Calavera, L. Fontana, and J. Frazelle, *Linux observability with BPF: Advanced Programming for performance analysis and Networking*. O'Reilly Media, Inc, 2020.

[47] R. Nair and T. Field, "Gapp: A fast profiler for detecting serialization bottlenecks in parallel linux applications," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '20. ACM, Apr. 2020, p. 257–264. [Online]. Available: http://dx.doi.org/10.1145/3358960.3379136

[48] T. V, V. R. Dixit, N. R. S, V. K. Gowda, S. K. Vasudevan, and S. Kalambur, "Misertrace: Kernel-level request tracing for microservice visibility," in *Companion of the 2022 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '22. ACM, Jul. 2022, pp. 77–80. [Online]. Available: http://dx.doi.org/10.1145/3491204.3527462

[49] "Helper function bpf_trace_printk." [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_trace_printk

[50] "printf(3) - linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man3/printf.3.html

[51] "Unix programmer's manual november 3, 1971," 1971. [Online]. Available: https://www.nokia.com/bell-labs/about/dennis-m-ritchie/1stEdman.html

[52] A. S. Tanenbaum, *Moderne Betriebssysteme*, 2014.

[53] R. E. Kálmán, "Contributions to the theory of optimal control," 1960. [Online]. Available: https://api.semanticscholar.org/CorpusID:845554

[54] S. Gill, "The diagnosis of mistakes in programmes on the edsac," *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, vol. 206, no. 1087, pp. 538–554, 1951. [Online]. Available: https://royalsocietypublishing.org/doi/abs/10.1098/rspa.1951.0087

[55] M. Kerrisk, "strace(1) — linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man1/strace.1.html

[56] C. V. Ravishankar and J. R. Goodman, "Cache implementation for multiple microprocessors," None. IEEE,New York, NY, 12 1982. [Online]. Available: https://www.osti.gov/biblio/5139336

[57] "gprof(1) - linux man page." [Online]. Available: https://linux.die.net/man/1/gprof

[58] M. Kerrisk, "ethtool(8) — linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man8/ethtool.8.html

[59] ——, "netstat(8) — linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man8/netstat.8.html

[60] ——, "ip(8) — linux manual page." [Online]. Available: https://man7.org/linux/man-pages/man8/ip.8.html

[61] A. Mavinakayanahalli, P. Panchamukhi, J. Keniston, A. Keshavamurthy, and M. Hiramatsu, 2006. [Online]. Available: https://www.kernel.org/doc/ols/2006/ols2006v2-pages-109-124.pdf

[62] K. Jim, P. Prasanna, and H. Masami, "Kernel probes (kprobes)." [Online]. Available: https://www.kernel.org/doc/html/v6.16/trace/kprobes.html

[63] K. Vamis, "Uprobe-tracer: Uprobe-based event tracing." [Online]. Available: https://docs.kernel.org/trace/uprobetracer.html

[64] K. Vamsi, "Linux kernel source code: kprobes.c," 2025, version 6.16. [Online]. Available: https://elixir.bootlin.com/linux/v6.16/source/kernel/kprobes.c

[65] W. Stallings, *Computer Organization and Architecture*. Pearson Education, 2018.

[66] T. L. Foundation, "Linux kernel source code: traps.c," 2025, version 4.2.0. [Online]. Available: https://elixir.bootlin.com/xen/v4.2.0/source/extras/mini-os/arch/x86/traps.c#L266

[67] Intel, "Intel 64 and ia-32 architectures software developer's manual," 2025. [Online]. Available: https://cdrdv2-public.intel.com/671447/325384-sdm-vol-3abcd.pdf

[68] B. Ivan, "ebpf overhead benchmark." [Online]. Available: https://github.com/cloudflare/ebpf_exporter/blob/master/benchmark/README.md

[69] P. Hübner, A. Hu, I. Peng, and S. Markidis, "Apple vs. oranges: Evaluating the apple silicon m-series socs for hpc performance and efficiency," 02 2025.

[70] S. Voyage, "Intel xeon e5410." [Online]. Available: https://siliconvoyage.com/cpus/intel-xeon-e5410

[71] R. Pi, "Raspberry pi 5." [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5

[72] J. Corbet, "Uprobes in 3.5." [Online]. Available: https://lwn.net/Articles/499190/

[73] S. Dronamraju, "Patch kprobes for 2.5.48." [Online]. Available: https://www.cs.helsinki.fi/linux/linux-kernel/2002-46/0267.html

[74] ——, "Linux kernel source code: uprobe.c," 2025, version 6.16.1. [Online]. Available: https://elixir.bootlin.com/linux/v6.16.1/source/kernel/events/uprobes.c

[75] ——, "Patch v4 0/13 uprobes v4." [Online]. Available: https://lkml.org/lkml/2010/5/18/307

[76] Y. Zheng, T. Yu, Y. Yang, Y. Hu, X. Lai, and A. Quinn, "bpftime: userspace ebpf runtime for uprobe, syscall and kernel-user interactions," 2023. [Online]. Available: https://arxiv.org/abs/2311.07923

[77] Intel, "Intel architecture software developer's manual volume 2: Instruction set reference," 2025. [Online]. Available: https://web.archive.org/web/20090219101735/http://download.intel.com/design/PentiumII/manuals/24319102.PDF

[78] J. Keniston, A. Mavinakayanahalli, P. Panchamukhi, and V. Prasad, "Ptrace, utrace, uprobes: Lightweight, dynamic tracing of user apps," pp. 215 – 224.

[79] Y. Zheng, T. Yu, Y. Yang, Y. Hu, X. Lai, and A. Quinn, "bpftime: Userspace ebpf runtime for observability, network, gpu & general extensions framework," 2023. [Online]. Available: https://github.com/eunomia-bpf/bpftime?tab=readme-ov-file#roadmap

[80] T. L. Foundation, "Linux kernel markers," 2008, zugriff am 19. August 2025. [Online]. Available: https://kernelnewbies.org/Linux_2_6_24#Linux_Kernel_Markers

[81] M. Desnoyers and M. Dagenais, "Lttng: Tracing across execution layers, from the hypervisor to user-space," vol. 1, pp. 101 – 106, 01 2008.

[82] M. Desnoyers, "Low-impact operating system tracing," Ph.D. dissertation, École Polytechnique de Montréal, 2009, zugriff am 19. August 2025. [Online]. Available: https://publications.polymtl.ca/206/1/2009_MathieuDesnoyers.pdf

[83] ——, "index : kernel/git/torvalds/linux.git," 2008. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=97e1c18e8d17bd87e1e383b2e9d9fc740332c8e2

[84] B. D.J., "Linux kernel source code: kprobes.c," 2025, version 6.16.1. [Online]. Available: https://elixir.bootlin.com/linux/v6.16.1/source/net/core/dev.c

[85] M. Desnoyers, "Using the linux kernel tracepoints," 2025, zugriff am 19. August 2025. [Online]. Available: https://docs.kernel.org/trace/tracepoints.html

[86] M. Desnoyers and M. Dagenais, "The lttng tracer: A low impact performance and behavior monitor for gnu/linux," in *Linux Symposium*, 2006, pp. 209–224, aRRAY(0x556c349d56f0). [Online]. Available: https://publications.polymtl.ca/23165/

[87] S. Rostedt, "Using the trace_event() macro (part 1)," 2010, zugriff am 19. August 2025. [Online]. Available: https://lwn.net/Articles/379903/

[88] J. Edge, "Minimizing instrumentation impacts," 2025, zugriff am 19. August 2025. [Online]. Available: https://lwn.net/Articles/365833/

[89] M. Gebai and M. Dagenais, "Survey and analysis of kernel and userspace tracers on linux: Design, implementation, and overhead," *ACM Computing Surveys*, vol. 51, pp. 1–33, 03 2018.

[90] H. Masami, "[patch -tip v7 00/11] kprobes: Kprobes jump optimization support," 2009. [Online]. Available: https://sourceware.org/pipermail/systemtap/2009q4/014718.html

[91] M. Fleming, "Using user-space tracepoints with bpf," 2018, zugriff am 19. August 2025. [Online]. Available: https://lwn.net/Articles/753601/

[92] B. M. Cantrill, M. W. Shapiro, and A. H. Leventhal, "Dynamic instrumentation of production systems," in *2004 USENIX Annual Technical Conference (USENIX ATC 04)*. Boston, MA: USENIX Association, Jun. 2004. [Online]. Available: https://www.usenix.org/conference/2004-usenix-annual-technical-conference/dynamic-instrumentation-production-systems

[93] M. Kerrisk, "perf_event_open(2) — linux manual page," 2025, zugriff am 19. August 2025. [Online]. Available: https://man7.org/linux/man-pages/man2/perf_event_open.2.html

[94] J. M. Aceituno, A. Guasque, P. Balbastre, J. Simó, and A. Crespo, "Hardware resources contention-aware scheduling of hard real-time multiprocessor systems," *Journal of Systems Architecture*, vol. 118, p. 102223, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762121001569

[95] S. Baruah, M. Bertogna, and G. Buttazzo, *The Three-Parameter Sporadic Tasks Model*. Cham: Springer International Publishing, 2015, pp. 95–102. [Online]. Available: https://doi.org/10.1007/978-3-319-08696-5_10

[96] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973. [Online]. Available: https://doi.org/10.1145/321738.321743

[97] M. J. Khojasteh, P. Tallapragada, J. Cortes, and M. Franceschetti, "Time-triggering versus event-triggering control over communication channels," 2017. [Online]. Available: https://arxiv.org/abs/1703.10744

[98] D. Meister, F. Dürr, and F. Allgöwer, "Shared network effects in time- versus event-triggered consensus of a single-integrator multi-agent system," 2023. [Online]. Available: https://arxiv.org/abs/2211.08156

[99] M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.

[100] "Crate libbpf-rs." [Online]. Available: https://docs.rs/libbpf-rs/latest/libbpf_rs/

[101] "Build scripts." [Online]. Available: https://doc.rust-lang.org/cargo/reference/build-scripts.html

[102] "Struct skeletonbuilder." [Online]. Available: https://docs.rs/libbpf-cargo/latest/libbpf_cargo/struct.SkeletonBuilder.html

[103] "Struct ringbufferbuilder." [Online]. Available: https://docs.rs/libbpf-rs/latest/libbpf_rs/struct.RingBufferBuilder.html

[104] C. Liu, B. Tak, and L. Wang, "Understanding performance of ebpf maps," in *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions*, ser. eBPF '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 9–15. [Online]. Available: https://doi.org/10.1145/3672197.3673430

[105] "Struct ringbuffer." [Online]. Available: https://docs.rs/libbpf-rs/latest/libbpf_rs/struct.RingBuffer.html#method.poll

[106] "repr(rust)." [Online]. Available: https://doc.rust-lang.org/nomicon/repr-rust.html

[107] "Derive." [Online]. Available: https://doc.rust-lang.org/reference/attributes/derive.html

[108] "Crate bytemuck." [Online]. Available: https://docs.rs/bytemuck/latest/bytemuck/

[109] J. Corbet, "Improving linux networking performance." [Online]. Available: https://lwn.net/Articles/629155/

[110] "Helper function bpf_ktime_get_ns." [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_ktime_get_ns/

[111] "Module time." [Online]. Available: https://doc.rust-lang.org/std/time/index.html

[112] "Struct systemtime." [Online]. Available: https://doc.rust-lang.org/std/time/struct.SystemTime.html

[113] "Application binary interface (abi)." [Online]. Available: https://doc.rust-lang.org/reference/abi.html?highlight=no_mangle#the-no_mangle-attribute

[114] "Keyword extern." [Online]. Available: https://doc.rust-lang.org/std/keyword.extern.html

[115] "Program types (linux)." [Online]. Available: https://docs.ebpf.io/linux/program-type/

[116] "Helper function bpf_ringbuf_submit." [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_ringbuf_submit/

[117] D. Mills, "Internet time synchronization: the network time protocol," *IEEE Transactions on Communications*, vol. 39, no. 10, pp. 1482–1493, 1991.

[118] D. Chefrour, "Evolution of network time synchronization towards nanoseconds accuracy: A survey," *Computer Communications*, vol. 191, pp. 26–35, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140366422001359

[119] "Ieee standard for a precision clock synchronization protocol for networked measurement and control systems," *IEEE Std 1588-2002*, pp. 1–154, 2002.

[120] "struct sk_buff." [Online]. Available: https://docs.kernel.org/networking/skbuff.html

[121] "Internet Protocol," RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/info/rfc791

[122] "Transmission Control Protocol," RFC 793, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/info/rfc793

[123] "ps(1) — linux manual page." [Online]. Available: https://www.man7.org/linux/man-pages/man1/ps.1.html

[124] "About iw." [Online]. Available: https://wireless.docs.kernel.org/en/latest/en/users/documentation/iw.html